CSE3001
Database Management Systems

Modules	Syllabus
Module-2	**Relational Models:** Structure of relational databases, Domains, Relations, Relational algebra – fundamental operators and syntax, selection, Projection, relational algebra queries, tuple relational calculus, set operations, renaming, Joins, Division, syntax. Operators, grouping and ungrouping, relational comparison. Codd's rules, Relational Schemas, Introduction to UML **Relational database model:** Logical view of data, keys, integrity rules. **Normalization:** 1NF, 2NF, 3NF, BCNF, Multi valued dependencies and Fourth Normal Form.

Introduction to Relational Database:

Relational Model:

- Collection of multiple **Relations or Tables**
- Tables consist of **Rows & Columns**
- This is primary data model for commercial purpose.

Advantages of Relational Data Model:

- Simple & easy to understand
- Simple data presentation
- Complex queries can be expressed in easy way

A Relation consists of:

- Relational Schema
- Relational Instance

Introduction to Relational Database:

Relational Schema:

- Defines column heads in a table
- Also specifies relation name, field name (column) and domain of each field. For e.g.

Student(S_ID: Number(10), Name: Char(30), Address: Varchar(30))

Relational Instance:

- It is a set of tuples (also called records) in which tuples have same no of fields as in relational schema
- Also called a relation or table

SID	Name	Address
101	Ajay	Bhopal
102	Amit	Ujjain
103	Ankit	Indore

Fields or Columns

Tuples or Records

Introduction to Relational Database:

SID	Name	Address
101	Ajay	Bhopal
102	Amit	Ujjain
103	Ankit	Indore
104	Ankush	Bhopal

← **Fields or Columns**

Tuples or Records

Degree of Relation:

Number of fields in a relation i.e. 3

Cardinality:

Number of tuples in a relation i.e. 4

Characteristics of Relational Database:

- Relation`s name should be distinct from all other relations in database
- Each cell in relation contains exactly one atomic value
- All values in an attribute are of same domain
- Column name in a relation should be distinct
- Each tuple should have distinct value
- Order of attribute & tuples has no significance

Codd`s Rules:

- Edgar F. Codd has proposed 12 rules for a relational database model.
- These rules are defined to understand that what are required for a database to be called as relational database.
  - Rule 1: Information Rule
  - Rule 2: Guaranteed Access Rule
  - Rule 3: Systematic treatment of NULL values
  - Rule 4: Dynamic online catalog based on relational model
  - Rule 5: Comprehensive data sub language rule
  - Rule 6: View Updating Rule
  - Rule 7: High level Insert, Update & Delete
  - Rule 8: Physical Data Independence Rule
  - Rule 9: Logical Data Independence Rule
  - Rule 10: Integrity Independence Rule
  - Rule 11: Distribution Independence
  - Rule 12: Non-Supervision Rule

Codd`s Rules:

Rule 1: Information Rule

The data stored in a database, may it be user data or metadata, must be a value of some table cell. Everything in a database must be stored in a table format.

Rule 2: Guaranteed Access Rule

Every single data element (value) is guaranteed to be accessible logically with a combination of table-name, primary-key (row value), and attribute-name (column value)

Rule 3: Systematic treatment of NULL values

This is a very important rule because a NULL can be interpreted as one the following – data is missing, data is not known, or data is not applicable.

Rule 4: Dynamic online catalog based on relational model

The structure description of the entire database must be stored in an online catalog, known as data dictionary, which can be accessed by authorized users.

Codd`s Rules:

Rule 5: Comprehensive Data Sub Language Rule

A database can only be accessed using a language having linear syntax that supports data definition, data manipulation, and transaction management operations.

Rule 6: View Updating Rule

All the views of a database, which can theoretically be updated, must also be updatable by the system.

Rule 7: High level Insert, Update & Delete

A database must support high-level insertion, updation, and deletion.

Rule 8: Physical Data Independence Rule

The data stored in a database must be independent of the applications that access the database. Any change in the physical structure of a database must not have any impact on how the data is being accessed by external applications.

Codd`s Rules:

Rule 9: Logical Data Independence Rule

The logical data in a database must be independent of its user's view (application). Any change in logical data must not affect the applications using it.

Rule 10: Integrity Independence Rule

All its integrity constraints can be independently modified without the need of any change in the application.

Rule 11: Distribution Independence

The end-user must not be able to see that the data is distributed over various locations. Users should always get the impression that the data is located at one site only.

Rule 12: Non-Supervision Rule

If a system has an interface that provides access to low-level records, then the interface must not be able to bypass security and integrity constraints.

Relational Algebra

- Procedural language-

  "What data is needed and how to find those data"

- Six basic operators
  - select
  - project
  - union
  - set difference
  - Cartesian product
  - rename
- The operators take one or more relations as inputs and give a new relation as a result.

Select Operation

- Notation: $\sigma_p(r)$
- *p* is called the selection predicate

- Defined as:

$$\sigma_p(\boldsymbol{r}) = \{t \mid t \in r \textbf{ and } p(t)\}$$

Where *p* is a formula in propositional calculus consisting of terms connected by : $\wedge$ (**and**), $\vee$ (**or**), $\neg$ (**not**)

Each term is one of: <attribute> *op* <attribute> or <constant>

where *op* is one of: =, $\neq$, >, $\geq$. <. $\leq$

- Example of selection:

$\sigma_{\textit{branch-name}=\text{“}\textit{Indore}\text{”}}(\textit{account})$

Select Operation – Example

- Relation *r*

A	*B*	*C*	*D*
α	α	1	7
α	β	5	7
β	β	12	3
β	β	23	10

- $\sigma_{A=B \wedge D > 5}(r)$

A	*B*	*C*	*D*
α	α	1	7
β	β	23	10

Project Operation

- Notation:

$$\Pi_{A1, A2, \ldots, Ak}\ (r)$$

  where A_1, A_2 are attribute names and r is a relation name.

- Duplicate rows removed from result, since relations are sets

- E.g. To eliminate the *branch-name* attribute of *account* from relation Account (account-number, balance, branch-name)

$$\Pi_{\textit{account-number, balance}}\ (\textit{account})$$

Project Operation – Example

- Relation *r*:

A	B	C
α	10	1
α	20	1
β	30	1
β	40	2

$\Pi_{A,C}(r)$

A	C
α	1
α	1
β	1
β	2

=

A	C
α	1
β	1
β	2

Example 1:

emp (ename, street, city)

works (ename, compname, salary)

company (compname, street, city)

Find name of those emp who works for ‘TCS’

SQL> Select ename from works where compname=‘TCS’;

$$\Pi_{ename} (\sigma_{compname='TCS'} (works))$$

Banking Example

customer (customer-name, customer-street, customer-only)

account (account-number, branch-name, balance)

loan (loan-number, branch-name, amount)

depositor (customer-name, account-number)

borrower (customer-name, loan-number)

Find all loans of over $1200

$$\sigma_{amount > 1200}(loan)$$

Display the loan number for each loan of an amount greater than $1200

$$\Pi_{loan\text{-}number}(\sigma_{amount > 1200}(loan))$$

Union Operation

- Notation: $r \cup s$
- Defined as:

$$r \cup s = \{t \mid t \in r \text{ or } t \in s\}$$

- For $r \cup s$ to be valid.
  1. *r, s* must have the *same arity* (same number of attributes)
  2. The attribute domains must be *compatible*

- E.g. to find all customers with either an account or a loan

$$\prod_{customer\text{-}name} (depositor) \cup \prod_{customer\text{-}name} (borrower)$$

Union Operation – Example

- Relations *r, s:*

A	B
α	1
α	2
β	1

r

A	B
α	2
β	3

s

$r \cup s$:

A	B
α	1
α	2
β	1
β	3

Banking Example Cont..

customer (customer-name, customer-street, customer-only)

account (account-number, branch-name, balance)

loan (loan-number, branch-name, amount)

depositor (customer-name, account-number)

borrower (customer-name, loan-number)

Display the names of all customers who have a loan, an account, or both, from the bank

$$\Pi_{customer\text{-}name}\ (borrower) \cup \Pi_{customer\text{-}name}\ (depositor)$$

Set-Intersection Operation

- Notation: $r \cap s$
- Defined as:

  $r \cap s = \{ t \mid t \in r \textbf{ and } t \in s \}$

- Assume:
  - r, s have the *same arity*
  - attributes of r and s are compatible
- Note: $r \cap s = r - (r - s)$

Set-Intersection Operation - Example

- Relation r, s:

A	B
α	1
α	2
β	1

r

A	B
α	2
β	3

s

- r ∩ s

A	B
α	2

Banking Example Cont..

customer (customer-name, customer-street, customer-only)

account (account-number, branch-name, balance)

loan (loan-number, branch-name, amount)

depositor (customer-name, account-number)

borrower (customer-name, loan-number)

Display the names of all customers who have a loan and an account at bank.

$$\Pi_{customer\text{-}name}\ (borrower) \cap \Pi_{customer\text{-}name}\ (depositor)$$

Set Difference Operation

- Notation $r - s$
- Defined as:

$$r - s = \{t \mid t \in r \textbf{ and } t \notin s\}$$

- Set differences must be taken between *compatible* relations.
  - r and s must have the *same arity*
  - attribute domains of r and s must be compatible

Set Difference Operation – Example

- Relations *r, s:*

A	B
α	1
α	2
β	1

r

A	B
α	2
β	3

s

r – s:

A	B
α	1
β	1

Example 1 Cont..:

emp (ename, street, city)

works (ename, compname, salary)

company (compname, street, city)

Display name of those emp who don`t work for 'TCS'

$$\prod_{ename}(works)) - \prod_{ename}(\sigma_{compname='TCS'}(works))$$

Cartesian-Product Operation

- Notation $r \times s$

- Defined as:

$$r \times s = \{t\, q \mid t \in r \textbf{ and } q \in s\}$$

Cartesian Product:

• To understand how SQL processes a join, it is important to understand the concept of the Cartesian product.

• A query that lists multiple tables in the FROM clause without a WHERE clause produces all possible combinations of rows from all tables.

•This result is called the *Cartesian product*.

```
select *
   from one, two;
```

Cartesian Product

Table One

X	A
1	a
4	d
2	b

3 rows X 3 rows

Table Two

X	B
2	x
3	y
5	v

Result Set

X	A	X	B
1	a	2	x
1	a	3	y
1	a	5	v
4	d	2	x
4	d	3	y
4	d	5	v
2	b	2	x
2	b	3	y
2	b	5	v

9 rows

Cartesian-Product Operation-Example

Relations *r, s*:

A	*B*
α	1
β	2

r

C	*D*	*E*
α	10	a
β	10	a
β	20	b
γ	10	b

s

r x *s*:

A	*B*	*C*	*D*	*E*
α	1	α	10	a
α	1	β	10	a
α	1	β	20	b
α	1	γ	10	b
β	2	α	10	a
β	2	β	10	a
β	2	β	20	b
β	2	γ	10	b

Composition of Operations

- Can build expressions using multiple operations
- Example: $\sigma_{A=C}(r \times s)$
- $r \times s$

A	B	C	D	E
α	1	α	10	a
α	1	β	10	a
α	1	β	20	b
α	1	γ	10	b
β	2	α	10	a
β	2	β	10	a
β	2	β	20	b
β	2	γ	10	b

- $\sigma_{A=C}(r \times s)$

A	B	C	D	E
α	1	α	10	a
β	2	β	20	a
β	2	β	20	b

Banking Example Cont..

customer (customer-name, customer-street, customer-only)

account (account-number, branch-name, balance)

loan (loan-number, branch-name, amount)

depositor (customer-name, account-number)

borrower (customer-name, loan-number)

Display the names of all customers who have a loan at 'M.P.Nagar' branch.

$$\Pi_{customer\text{-}name}\ (\sigma_{branch\text{-}name=\text{“M.P.Nagar”}\ \wedge\ borrower.loan\text{-}number\ =loan.loan\text{-}number}(borrower\ x\ loan))$$

Display the names of all customers who have a loan at the 'New Market' branch but do not have an account at any branch of the bank.

$$\Pi_{customer\text{-}name}\ (\sigma_{branch\text{-}name\ =\ \text{“New Market”}\ \wedge\ borrower.loan\text{-}number\ =\ loan.loan\text{-}number}(borrower\ x\ loan)))\ -\ \Pi_{customer\text{-}name}(depositor)$$

Example 1:

emp1 (ssn, name)

sal1 (ssn, salary) // ssn referencing to emp

works1 (project#, ssn) // project# referencing to project & ssn referencing to emp

proj1 (project#, project_name, location)

Q. Display the projects name at Delhi.

SQL> select project_name from proj1 where location='Delhi';

$\Pi_{project_name}\ (\sigma_{location="Delhi"}(proj1))$

Q. Retrieve the name and ssn of those employees working on project no 'P101'.

SQL> select emp1.ssn, emp1.name from emp1,works1 where emp1.ssn=works1.ssn and project#='P01';

$\Pi_{emp1.ssn,emp1.name}\ (\sigma_{emp1.ssn=works1.ssn\ \wedge\ project\#='P01'}(emp1 \times works1))$

$\Pi_{emp1.ssn,emp1.name}\ (\sigma_{project\#='P01'}(emp1 \bowtie works1))$

Example 1 Cont..:

emp1 (ssn, name)

sal1 (ssn, salary) // ssn referencing to emp

works1 (project#, ssn) // project# referencing to project & ssn referencing to emp

proj1 (project#, project_name, location)

Q. Find the project name of employees whose salary is greater than 50000.

SQL> select project_name from sal1,works1,proj1 where salary>50000 and proj1.project# =works1.project# and works1.ssn=sal1.ssn;

$\Pi_{project_name}\ (\sigma_{proj1.project\#=works1.project\#\ \wedge\ works1.ssn=sal1.ssn\ \wedge\ salary>50000}\ (sal1 \times works1 \times proj1))$

$\Pi_{project_name}\ (\sigma_{salary>50000}\ (sal1 \bowtie works1 \bowtie proj1))$

Example 1 Cont..:

emp1 (ssn, name)

sal1 (ssn, salary) // ssn referencing to emp

works1 (project#, ssn) // project# referencing to project & ssn referencing to emp

proj1 (project#, project_name, location)

Q. Find the employees name working on project no P201 and salary>45000.

SQL> select emp1.name from emp1,works1,sal1 where emp1.ssn=works1.ssn andworks1.ssn=sal1.ssn and project#='P201' and salary>45000;

$\Pi_{emp1.name}\ (\sigma_{emp1.ssn=works1.ssn\ \wedge\ works1.ssn=sal1.ssn\ \wedge\ project\#='P201'\ \wedge\ salary>45000}\ (emp1 \times works1 \times sal1))$

$\Pi_{emp1.name}\ (\sigma_{project\#='P201'\ \wedge\ salary>45000}\ (emp1 \bowtie works1 \bowtie sal1))$

Example 3:

emp (ename, street, city)

works (ename, compname, salary)

company (compname, street, city)

Find name of those emp who works for ‘TCS’

SQL> Select ename from works where compname=‘TCS’;

$$\Pi_{ename} (\sigma_{compname='TCS'} (works))$$

Find name & street address of those emp who works for ‘TCS’

SQL> Select emp.ename,emp.street from emp,works where compname= ‘TCS’ and emp.ename=works.ename;

$$\Pi_{emp.ename,emp.street} (\sigma_{compname='TCS' \wedge emp.ename=works.ename}(emp \times works))$$

Example 3:

emp (ename, street, city)

works (ename, compname, salary)

company (compname, street, city)

Find name of those emp who don`t work for 'TCS'

$$\prod_{ename}(works)) - \prod_{ename}(\sigma_{compname='TCS'}(works))$$

Find name of those emp who live in the same street and city as the company for which they work

$$\prod_{emp.ename}(\sigma_{emp.ename=works.ename \wedge works.compname=company.compname \wedge emp.street=company.street \wedge emp.city=works.city}(\text{emp x } works \text{ x } company))$$

Example 4:

Consider the following relational database schema consisting of the four relation schemas:

- **passenger** (pid, pname, pgender, pcity)
- **agency** (aid, aname, acity)
- **flight** (fid, fdate, time, src, dest)
- **booking** (pid, aid, fid, fdate)

Answer the following questions using relational algebra queries;

a) Get the complete details of all flights to New Delhi.

b) Get the details about all flights from Chennai to New Delhi.

c) Find only the flight numbers for passenger with pid 123 for flights to Chennai before 06/11/2020.

d) Find the passenger names for passengers who have bookings on at least one flight.

Example 4:

e) Find the passenger names for those who do not have any bookings in any flights.

f) Find the agency names for agencies that located in the same city as passenger with passenger id 123.

g) Get the details of flights that are scheduled on both dates 01/12/2020 and 02/12/2020 at 16:00 hours.

h) Get the details of flights that are scheduled on either of the dates 01/12/2020 or 02/12/2020 or both at 16:00 hours.

i) Find the agency names for agencies who do not have any bookings for passenger with id 123.

j) Find the details of all male passengers who are associated with Jet agency.

passenger (pid, pname, pgender, pcity)

agency (aid, aname, acity)

flight (fid, fdate, time, src, dest)

booking (pid, aid, fid, fdate)

a) Get the complete details of all flights to New Delhi.

$$\sigma_{destination = \text{“New Delhi”}} (\text{flight})$$

b) Get the details about all flights from Chennai to New Delhi.

$$\sigma_{src = \text{“Chennai”} \wedge dest = \text{“New Delhi”}} (\text{flight})$$

c) Find only the flight numbers for passenger with pid 123 for flights to Chennai before 06/11/2020.

$$\Pi_{fid} (\sigma_{pid = 123} (\text{booking}) \bowtie \sigma_{dest = \text{“Chennai”} \wedge fdate < 06/11/2020} (\text{flight}))$$

d) Find the passenger names for passengers who have bookings on at least one flight.

$$\Pi_{pname} (\text{passenger} \bowtie \text{booking})$$

passenger (pid, pname, pgender, pcity)

agency (aid, aname, acity)

flight (fid, fdate, time, src, dest)

booking (pid, aid, fid, fdate)

e) Find the passenger names for those who do not have any bookings in any flights.

$$\Pi_{pname}((\Pi_{pid}(\text{passenger}) - \Pi_{pid}(\text{booking})) \bowtie \text{passenger})$$

f) Find the agency names for agencies that located in the same city as passenger with passenger id 123.

$$\Pi_{aname}(\text{agency} \bowtie_{acity = pcity}(\sigma_{pid = 123}(\text{passenger})))$$

g) Get the details of flights that are scheduled on both dates 01/12/2020 and 02/12/2020 at 16:00 hours.

$$(\sigma_{fdate = 01/12/2020 \wedge time = 16:00}(\text{flight})) \cap (\sigma_{fdate = 02/12/2020 \wedge time = 16:00}(\text{flight}))$$

passenger (pid, pname, pgender, pcity)

agency (aid, aname, acity)

flight (fid, fdate, time, src, dest)

booking (pid, aid, fid, fdate)

h) Get the details of flights that are scheduled on either of the dates 01/12/2020 or 02/12/2020 or both at 16:00 hours.

$(\sigma_{fdate = 01/12/2020 \wedge time = 16:00}(\text{flight})) \cup (\sigma_{fdate = 02/12/2020 \wedge time = 16:00}(\text{flight}))$

i) Find the agency names for agencies who do not have any bookings for passenger with id 123.

$\Pi_{aname}(\text{agency} \bowtie (\Pi_{\text{aid}}(\text{agency}) - \Pi_{\text{aid}}(\sigma_{pid = 123}(\text{booking}))))$

j) Find the details of all male passengers who are associated with Jet agency.

$\Pi_{passengers.pid, pname, pcity}(\sigma_{pgender = \text{“Male”} \wedge aname = \text{‘Jet’}}(\text{passengers} \bowtie \text{booking} \bowtie \text{agency}))$

Extended Relational Algebra Operations

Extended Relational-Algebra-Operations

- **Rename Operation**
- **Division Operation**
- **Aggregate Functions**
- **Generalized Projection**

Rename Operation

- Allows us to refer to a relation by more than one name.

  Example:

  $$\rho_X(E)$$

  returns the expression *E* under the name *X*

- If a relational-algebra expression *E* has arity *n*, then

  $$\rho_{X(A1, A2, \ldots, An)}(E)$$

  returns the result of expression *E* under the name *X*, and with the attributes renamed to *A1, A2, …., An*.

Division Operation

$$r \div s$$

- Suited to queries that include the phrase “for all”.

- Let *r* and *s* be relations on schemas R and S respectively where
  - $R = (A_1, \ldots, A_m, B_1, \ldots, B_n)$
  - $S = (B_1, \ldots, B_n)$

The result of r ÷ s is a relation on schema

$R - S = (A_1, \ldots, A_m)$

Division Operation – Example

Relations *r, s*:

A	B
α	1
α	2
α	3
β	1
γ	1
δ	1
δ	3
δ	4
$\in$	6
$\in$	1
β	2

r

B
1
2

s

r ÷ *s*:

A
α
β

Another Division Example

Relations *r, s*:

A	*B*	*C*	*D*	*E*
α	a	α	a	*1*
α	a	γ	a	*1*
α	a	γ	b	*1*
β	a	γ	a	*1*
β	a	γ	b	*3*
γ	a	γ	a	*1*
γ	a	γ	b	*1*
γ	a	β	b	*1*

r

D	*E*
a	*1*
b	*1*

s

a,b,c column which has both the d,e as a=1 and the b=1 only those rows are output

r ÷ *s*:

A	*B*	*C*
α	a	γ
γ	a	γ

Aggregate Functions and Operations

- **Aggregation function** takes a collection of values and returns a single value as a result.

  avg: average value
  min: minimum value
  max: maximum value
  sum: sum of values
  count: number of values

- **Aggregate operation** in relational algebra

$$_{G1, G2, \ldots, Gn}\mathcal{g}_{F1(A1), F2(A2), \ldots, Fn(An)}(E)$$

  - E is any relational-algebra expression
  - $G_1, G_2 \ldots, G_n$ is a list of attributes on which to group (can be empty)
  - Each F_i is an aggregate function
  - Each A_i is an attribute name

Aggregate Operation – Example

- Relation *r*:

A	*B*	*C*
α	α	7
α	β	7
β	β	3
β	β	10

$$\mathcal{G}_{\mathbf{sum(c)}}(\mathrm{r})$$

sum-C
27

Aggregate Operation – Example

- Relation *account* grouped by *branch-name*:

branch-name	*account-number*	*balance*
Perryridge	A-102	400
Perryridge	A-201	900
Brighton	A-217	750
Brighton	A-215	750
Redwood	A-222	700

$$branch\text{-}name\ \mathcal{G}\ sum(balance)\ (account)$$

branch-name	*balance*
Perryridge	1300
Brighton	1500
Redwood	700

Aggregate Functions (Cont.)

- Result of aggregation does not have a name
  - Can use rename operation to give it a name
  - For convenience, we permit renaming as part of aggregate operation

$$branch\text{-}name\ \mathcal{G}\ \textbf{sum}(balance)\ \textbf{as}\ sum\text{-}balance\ (account)$$

Generalized Projection

- Extends the projection operation by allowing arithmetic functions to be used in the projection list.

$$\Pi_{F1, F2, \ldots, Fn}(E)$$

- E is any relational-algebra expression
- Each of F_1, F_2, …, F_n are are arithmetic expressions involving constants and attributes in the schema of E.

- Given relation *credit-info(customer-name, limit, credit-balance),* find how much more each person can spend:

$$\Pi_{customer\text{-}name,\ limit - credit\text{-}balance}\ (credit\text{-}info)$$

Relational Calculus

- Tuple Relational Calculus is a **non-procedural query language** unlike relational algebra.
- Tuple Calculus provides only the description of the query not the methods to solve it.
- Thus, it explains what to do but not how to do.
- Specifies, what is to be retrieved rather than how to retrieved.

2 Types:

i. Tuple Relational Calculus

ii. Domain Relational Calculus

Tuple Relational Calculus

- A nonprocedural query language, where each query is of the form

$$\{t \mid P(t)\}$$

- It is the set of all tuples *t,* such that predicate *P* is true for *t*
- *t* is a *tuple variable*, *t*[*A*] denotes the value of tuple *t* on attribute *A*
- $t \in r$ denotes that tuple *t* is in relation *r*
- *P* is a *formula* similar to that of the predicate calculus

Tuple Relational Calculus Example:

emp(fname, ssn, dno, salary)

dept(dname, dno, mgrssn)

emp

fname	ssn	dno	salary
Ajay	1	D5	30000
Bhanu	3	D5	40000
Ram	9	D1	35000
Namit	6	D4	30000
Jayesh	2	D3	45000

dept

dname	dno	mgrssn
Research	D5	3
Admin	D4	6
HR	D3	-

Tuple Relational Calculus Example:

emp(ename, ssn, dno, salary)

dept(dname, dno, mgrssn)

1. Find name of all employees using tuple relational calculus.

$\{t \mid \exists e \in emp\ (t[ename]=e[ename])\}$

2. Find name of all employees whose salary more than 35000.

$\{t \mid \exists e \in emp\ (t[ename]=e[ename] \wedge e[salary]>35000)\}$

3. Find name of employees working for dept no D5.

$\{t \mid \exists e \in emp\ (t[ename]=e[ename] \wedge e[dno] = \text{'D5'})\}$

Tuple Relational Calculus Example:

emp(ename, ssn, dno, salary)

dept(dname, dno, mgrssn)

4. Find employee name who are manager.

$$\{t \mid \exists e \in emp\ (t[ename]=e[ename] \wedge \exists d \in dept\ (e[ssn] =d[mgrssn])\}$$

5. Find name of employees along with dept name for which they work.

$$\{t \mid \exists e \in emp \wedge \exists d \in dept\ (t[ename]=e[ename] \wedge t[dname]=d[dname] \wedge e[dno] =d[dno])\}$$

Domain Relational Calculus

- A nonprocedural query language equivalent in power to the tuple relational calculus
- Each query is an expression of the form:

$$\{ < x_1, x_2, \ldots, x_n > \mid P(x_1, x_2, \ldots, x_n)\}$$

  - $x_1, x_2, \ldots, x_n$ represent domain variables
  - P represents a formula similar to that of the predicate calculus

Domain Relational Calculus Example:

emp(fname, lname, ssn, dno, salary)

dept(dname, dno, mgrssn)

emp

p	q	r	s	t
fname	**lname**	**ssn**	**dno**	**salary**
Ajay	Kumar	1	D5	30000
Bhanu	Singh	3	D5	40000
Ram	Dubey	9	D1	35000
Namit	Sharma	6	D4	30000
Jayesh	Shukla	2	D3	45000

dept

l	m	n
dname	**dno**	**mgrssn**
Research	D5	3
Admin	D4	6
HR	D3	-

Domain Relational Calculus Example:

p q r s t
emp(fname, lname, ssn, dno, salary)

dept(dname, dno, mgrssn)
l m n

1. Find first name, last name of all employees using domain relational calculus.

$$\{<p,q> | <p,q,r,s,t> \in emp\}$$

2. Find name of all employees whose salary more than 35000.

$$\{<p,q> | <p,q,r,s,t> \in emp \wedge t>35000\}$$

Domain Relational Calculus Example:

p q r s t
emp(fname, lname, ssn, dno, salary)

dept(dname, dno, mgrssn)
l m n

3. Find all details of employees working for dept no D5.

$$\{<p,q,r,s,t> | <p,q,r,s,t> \in emp \wedge s = \text{'D5'}\}$$

4. Find employee name who are manager.

$$\{<p,q> | \exists r, n\ (<p,q,r,s,t> \in emp \wedge <l,m,n> \in dept \wedge r=n)\}$$

5. Find all details of employees along with dept name for which they work.

$$\{<p,q,r,s,t,l> | \exists s, m\ (<p,q,r,s,t> \in emp \wedge <l,m,n> \in dept \wedge s=m)\}$$

Banking Example

- *customer (customer-name, customer-street, customer-city)*
- *account (account-number, branch-name, balance)*
- *loan (loan-number, branch-name, amount)*
- *depositor (customer-name, account-number)*
- *borrower (customer-name, loan-number)*

Find names of all customers having a loan, an account or both at the bank.

Find names of all customers who have a loan and an account at the bank

Find the loan number for each loan of an amount greater than $1200

Find the *loan-number, branch-name and amount* for loans of over $1200

Find the names of all customers who have a loan of over $1200

Banking Example

- *customer (customer-name, customer-street, customer-city)*
- *account (account-number, branch-name, balance)*
- *loan (loan-number, branch-name, amount)*
- *depositor (customer-name, account-number)*
- *borrower (customer-name, loan-number)*

Find names of all customers having a loan, an account or both at the bank.

$$\{t \mid \exists b \in borrower(t[customer\text{-}name] = b[customer\text{-}name]) \vee \exists d \in depositor(t[customer\text{-}name] = d[customer\text{-}name])\}$$

Find names of all customers who have a loan and an account at the bank

$$\{t \mid \exists b \in borrower(t[customer\text{-}name] = b[customer\text{-}name]) \wedge \exists d \in depositor(t[customer\text{-}name] = d[customer\text{-}name])\}$$

Banking Example

- *customer (customer-name, customer-street, customer-city)*
- *account (account-number, branch-name, balance)*
- *loan (loan-number, branch-name, amount)*
- *depositor (customer-name, account-number)*
- *borrower (customer-name, loan-number)*

Find the loan number for each loan of an amount greater than $1200

TRC- $\{t \mid \exists l \in loan\ (t[loan\text{-}number] = l[loan\text{-}number] \wedge l[amount] > 1200)\}$

DRC- $\{\langle ln \rangle \mid \langle ln, bn, amt \rangle \in loan \wedge amt > 1200\}$

Banking Example

- *customer (customer-name, customer-street, customer-city)*
- *account (account-number, branch-name, balance)*
- *loan (loan-number, branch-name, amount)*
- *depositor (customer-name, account-number)*
- *borrower (customer-name, loan-number)*

Find the *loan-number, branch-name and amount* for loans of over $1200

TRC- $\{t \mid \exists l \in loan\ (t[loan\text{-}number] = l[loan\text{-}number] \wedge t[branch\text{-}name] = l[branch\text{-}name] \wedge t[amount] = l[amount] \wedge l[amount] > 1200)\}$

DRC- $\{< l, b, a > \mid < l, b, a > \in loan \wedge a > 1200\}$

Banking Example

- *customer (customer-name, customer-street, customer-city)*
- *account (account-number, branch-name, balance)*
- *loan (loan-number, branch-name, amount)*
- *depositor (customer-name, account-number)*
- *borrower (customer-name, loan-number)*

Find names of all customers having a loan at the Perryridge branch

TRC- $\{t \mid \exists b \in borrower\ (t[customer\text{-}name] = b[customer\text{-}name] \wedge \exists l \in loan\ (l[branch\text{-}name] = \text{“Perryridge”} \wedge l[loan\text{-}number] = b[loan\text{-}number]))\}$

DRC- $\{<c> \mid \exists ln, l, b\ (<c, ln> \in borrower \wedge <l, b, a> \in loan \wedge b = \text{“Perryridge”} \wedge ln = l)\}$

Banking Example

- *customer (customer-name, customer-street, customer-city)*
- *account (account-number, branch-name, balance)*
- *loan (loan-number, branch-name, amount)*
- *depositor (customer-name, account-number)*
- *borrower (customer-name, loan-number)*

Find the names of all customers who have a loan of over $1200

TRC- $\{t \mid \exists b \in borrower\ (t[customer\text{-}name] = b[customer\text{-}name] \wedge \exists l \in loan\ (l[amount] > 1200 \wedge l[loan\text{-}number] = b[loan\text{-}number]))\}$

DRC- $\{<c> \mid \exists ln, l, a\ (<c, ln> \in borrower \wedge <l, b, a> \in loan \wedge ln=l \wedge a > 1200)\}$

Relational Calculus Example:

Let the following relation schemas be given:

$$R = (A, B, C)$$

$$S = (D, E, F)$$

Let relations r(R) and s(S) be given. Give an expression in the tuple relational calculus that is equivalent to each of the following:

a. $\Pi_A^{(r)}$ $\{t \mid \exists\, r \in R\ (t[A] = r[A])\}$

b. $\sigma_{B = 17}^{(r)}$ $\{t \mid r \in R \wedge r[B] = 17\}$

c. $\Pi_{A,F}(\sigma_{C = D}^{(r \times s)})$ $\{t \mid \exists\, r \in R\ \exists\, s \in S\ (t[A] = r[A] \wedge t[F] = s[F] \wedge r[C] = s[D]$

d. $r \times s$ $\{t \mid \exists\, r \in R\ \exists\, s \in S\ (t[A] = r[A] \wedge t[B] = r[B] \wedge t[C] = r[C] \wedge t[D] = s[D] \wedge t[E] = s[E] \wedge t[F] = s[F])\}$

Relational Calculus Example:

Let R = (A, B, C), and let r1 and r2 both be relations on schema R. Give an expression in the domain relational calculus that is equivalent to each of the following:

a. $\Pi_A^{(r1)}$ $\{< a > | < a, b, c > \in r1\}$

b. $\sigma_{B = 17}^{(r1)}$ $\{< a, b, c > | < a, b, c > \in r1 \wedge b = 17\}$

c. $r1 \cup r2$ $\{< a, b, c > | < a, b, c > \in r1 \vee < a, b, c > \in r2\}$

d. $r1 \cap r2$ $\{< a, b, c > | < a, b, c > \in r1 \wedge < a, b, c > \in r2\}$

f. $\Pi_{A,B}^{(r1)} \bowtie \Pi_{B,C}^{(r2)}$ $\{< a, p, c > | \exists p, q (< a, p, c > \in r1 \wedge < a, q, c > \in r2 \wedge p=q)$

Identification of KEYs in DBMS?

Let`s consider a Relation :

STUDENT

SID	ROLL NO	NAME	ADDRESS	PHONE NO	AGE
S1	101	AJAY	BHOPAL	12345	21
S2	102	AMAN	UJJAIN	45678	22
S6	106	AMIT	BHOPAL		20
S8	108	AMIT	INDORE	78910	21
S9	109	ANKIT	UJJAIN	10234	23

Possible Keys are –

{SID and ROLL NO}

Types of KEYs in DBMS?

There are different types of Keys in DBMS and each key has it's different functionality:

- Super Key
- Candidate Key
- Primary Key
- Alternate Key/ Secondary Key
- Foreign Key

Super Key-

- A super key is a group of single or multiple keys which identifies rows in a table.
- A Super key may have additional attributes that are not needed for unique identification.

SID	ROLL NO	NAME	ADDRESS	PHONE NO
S1	101	AJAY	BHOPAL	12345
S2	102	AMAN	UJJAIN	45678
S6	106	AMIT	BHOPAL	
S8	108	AMIT	INDORE	78910
S9	109	ANKIT	UJJAIN	10234

A Relation may have ‘N’ numbers of super keys

ATTRIBUTE	SUPER KEY
SID	YES
ROLL NO	YES
NAME	NO
ADDRESS	NO
PHONE NO	NO
SID, ROLLNO	YES
NAME, ADD	NO
ADD, PHONE	NO
ROLLNO, ADD	YES
SID, NAME,P PHONE	YES

Candidate Key-

- Minimal of Super Key is called Candidate Key.
- A Super key whose proper subset is not a super key is known as Candidate key.

SID	ROLL NO	NAME	ADDRESS	PHONE NO
S1	101	AJAY	BHOPAL	12345
S2	102	AMAN	UJJAIN	45678
S6	106	AMIT	BHOPAL	
S8	108	AMIT	INDORE	78910
S9	109	ANKIT	UJJAIN	10234

	SUPER KEY	PROPER SUBSET	CANDIDATE KEY
SID	YES	Ø	YES
ROLL NO	YES	Ø	YES
SID, ROLL NO	YES	{SID, ROLLNO}	NO
NAME	NO	Ø	NO
SID, NAME	YES	{SID, NAME}	NO
NAME, PHONENO	NO	{NAME, PHONE}	NO
ROLLNO,ADD	YES	{ROLLNO, ADD}	NO

Primary Key-

- A primary key is a candidate key that the database designer selects while designing the database.
- Chosen Candidate key is Primary Key.
- There is only one primary key in a relation.
- Remaining candidate keys are called alternate keys or secondary keys.

SID	ROLL NO	NAME	ADDRESS	PHONE NO
S1	101	AJAY	BHOPAL	12345
S2	102	AMAN	UJJAIN	45678
S6	106	AMIT	BHOPAL	
S8	108	AMIT	INDORE	78910
S9	109	ANKIT	UJJAIN	10234

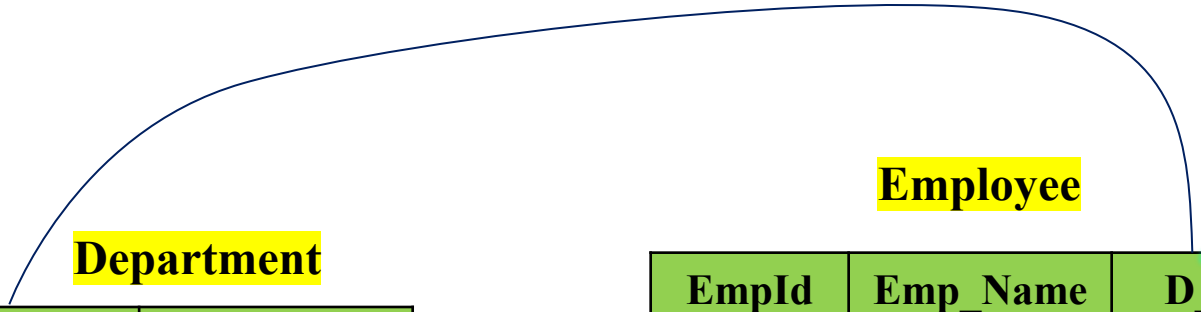
Candidate Keys are- {SID, ROLL NO}

Primary Key- either SID
or ROLL NO

Foreign Key-

- Foreign key is a concept of "Referential Integrity".
- It requires at least two relations (tables).
- One is '**Master or Parent Table**' & another is '**Slave or Child Table**"
- Foreign key is created in slave table & it always refers to the Primary Key of master table.
- If Master table doesn't contain any primary key then we can't take reference of that table. So it is always required to have primary key in master table.
- It is not necessary to have primary key in slave table.
- We may have N numbers of foreign key in slave table. Even we can create all columns as foreign key.
- It is not necessary to have same name of foreign key (in slave table) & primary key (in master table).
- But their data type must be same.

Foreign Key-



Department

Dept_Id	Dept_Name
D01	Admin
D02	Finance
D03	HR

(Master Table)

Employee

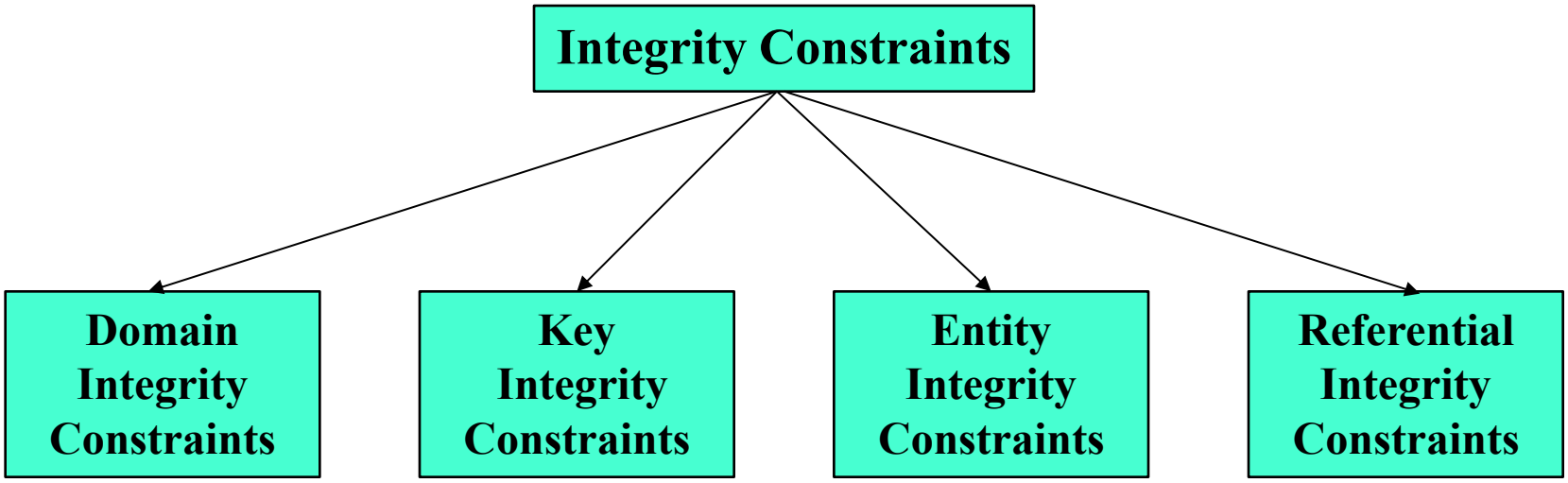
Foreign Key

EmpId	Emp_Name	D_Id	
1622	Aman	D03	
1625	Ankit	D01	
1631	Ankush	D03	
1637	Ayush	D02	
1639	Ajay	D04	Error: Parent key not found
1622	Aman	D01	
1601	Arun	-	
1613	Aarav	D07	Error: Parent key not found
1625	Ankit	D02	

(Slave Table)

Integrity Constraints:

- Integrity constraints are set of rules. They are used to maintain the quality of information.
- Integrity constraints ensure that the data insertion, updating, and other processes have to be performed in such a way that data integrity is not affected.
- Thus, integrity constraint is used to guard against accidental damage to the database.



1. Domain Constraints:

- Domain constraints can be defined as the definition of a valid set of values for an attribute.
- **Domain constraint** ensures that the value associated with an attribute is justifying its domain.
- The data type of domain includes string, character, integer, time, date, currency, etc.

ID	NAME	SEMENSTER	AGE
1000	Tom	1st	17
1001	Johnson	2nd	24
1002	Leonardo	5th	21
1003	Kate	3rd	19
1004	Morgan	8th	A

Not allowed, because AGE is an integer attribute

Now, there are three constraints which we can study under domain constraint-

- Not Null constraint
- Default constraint
- Check Clause constraint

Domain Constraints Cont..:

a. Not Null Constraint-

- In a relation, there are some attributes which need not to be null.
- By specifying an attribute to be '***Not Null***'**,** we restrict the domain of that attribute for not accepting the null values.
- Consider a student tuple that has a null value in it's 'name' attribute. In this case, we are storing information about an unknown student.
- So, in these cases, we have to particularly specify **not null constraint** for the specific attribute in a relation.

Let us see how do we specify an attribute to be not null-

create table *Student (Student_id* ***varchar (5)*** *,name* ***varchar (20) not null****,*
depart_name ***varchar (20)****);*

Domain Constraints Cont..:

b. Default Value Constraint-

- Using **default value constraint**, we are able to set a default value for an attribute.
- In case if we don't specify any value for an attribute on which default constraint is specified, it holds the specified default value.

For example:

create table *instructor (instructor_id* ***varchar (5)****,*
name ***varchar (20) not null****,*
depart_name ***varchar (5)****,*
salary numeric (8,2) ***default 0****);*

- This command specifies, if no value is provided for the salary attribute then its value is set to be 0.

Domain Constraints Cont..:

c. Check Clause Constraint-

- The **check clause constraint** ensures that when a new tuple is inserted in relation it must satisfy the predicate specified in the check clause.

Let's see an example of the check clause:

create table *Student (Student_id* ***varchar (5)*** *,*
name ***varchar (20) not null****,*
dept_name ***varchar (20),***
primary key *(Student_id),*
check *(dept_name in('CS', 'IT', 'EC', 'ME')));*

- According to the SQL standard, the predicate that is placed inside the check clause can be a **subquery**.

2. Key Constraints:

Primary key constraints:

- A primary key always contains ***Unique*** & ***Not Null*** value in a relation.

create table *Student (Student_id* ***varchar (5)*** *,*
name ***varchar (20) not null****,*
depart_name ***varchar (20),***
primary key *(Student_id));*

Unique key constraints:

- It is similar to a primary key but its value can store ***Null*** values also.
- But the values which are stored in that attribute should be ***Unique***.

create table *Student (Student_id* ***varchar (5) primary key*** *,*
name ***varchar (20) not null****,*
depart_name ***varchar (20),***
phone_no ***number(10) unique)****;*

3. Entity Constraints:

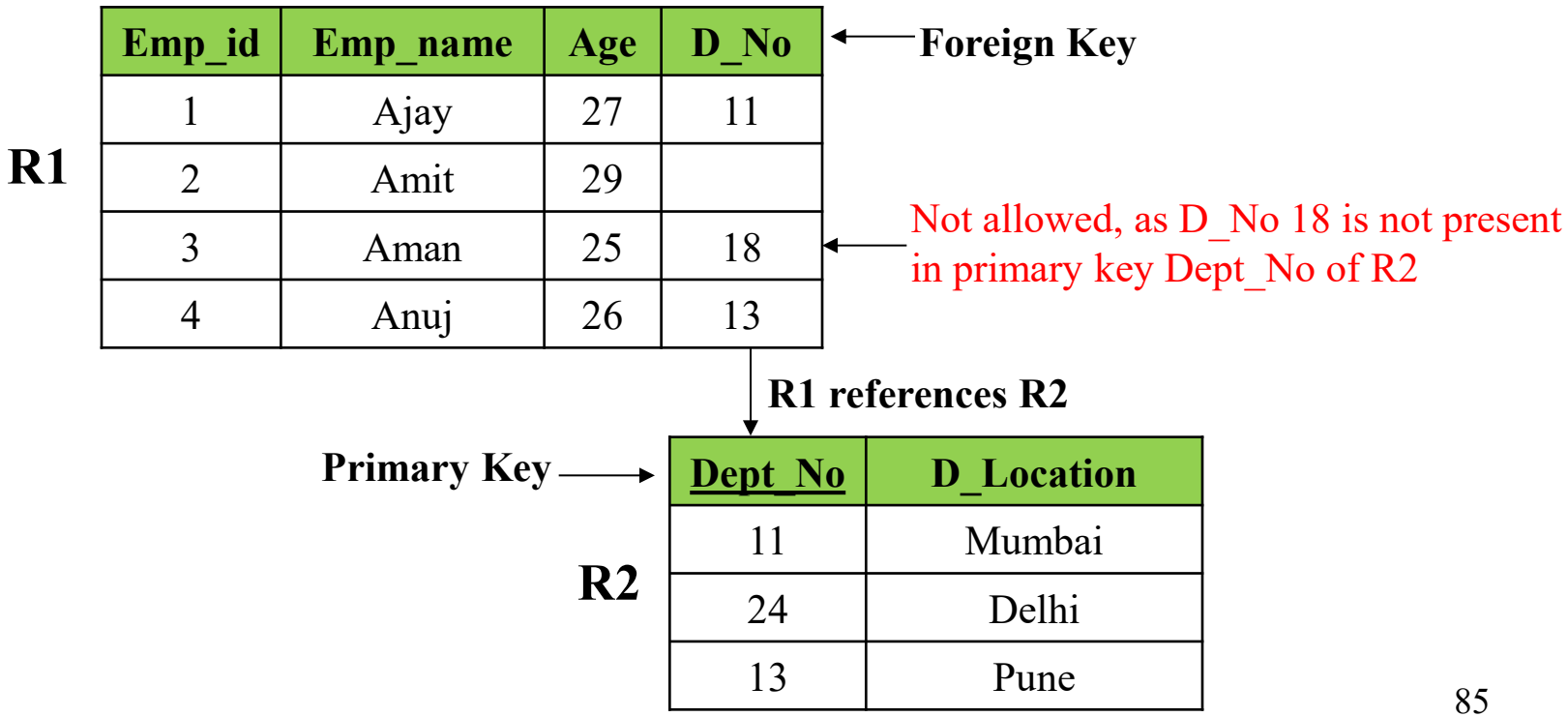
- Entity integrity constraint ensures that the primary key attribute in a relation, should not accept a null value.
- This is because the primary key attribute value uniquely defines an entity in a relation.

create table *Student (Student_id* ***varchar (5)*** *,*
name ***varchar (20) not null****,*
depart_name ***varchar (20),***
primary key *(Student_id));*

- Whenever we declare any attribute in relation as the primary key, it doesn't necessary to specify it **explicitly** to be not null.

4. Referential Integrity Constraints:

- This is the concept of foreign key.
- A referential integrity constraint is specified between two tables.
- In the Referential integrity constraints, if a foreign key in relation R1 refers to the Primary Key of relation R2, then every value of the Foreign Key in R1 must be null or be available in R2.



Referential Integrity Constraints Cont..:

Concept of Referential Integrity says that-

"if there are two relations R1 & R2 having primary key K1 & K2 respectively and a subset α in R2 referencing to primary key K1 in R1 then for every tuple t2 in R2, there must be a tuple t1 in R1such that:

$$t2[\alpha] = t1[k1]$$

This concept is called Referential Integrity"

R1

	K1	
t1	ABC	

R2

	K2	α	β
		ABC	
t2			

Functional Dependency:

- Functional dependency in DBMS, as the name suggests is a relationship between attributes of a table dependent on each other.
- Introduced by E. F. Codd, it helps in preventing data redundancy and gets to know about bad designs.

Let us consider R is a relation with attributes A and B.

Then the following will represent the functional dependency between attributes with an arrow sign-

A -> B

B- Functionally dependent on A

A- Determinant set

B- Dependent attribute

Functional Dependency Cont..:

- Functional dependency says that if two tuples have same values for attributes A1, A2,..., An, then those two tuples must have same values for attributes B1, B2, ..., Bn.

Let us consider R is a relation with attributes A and B.

A->B is called a functional dependency,

if there are two tuples t1 & t2 such that t1[A]=t2[A] then t1[B]=t2[B] must hold. (This concept is called functional dependency)

A	B	C	D
a1	b1	c1	d1
a1	b1	c2	d1
a2	b2	c3	d1
a2	b2	c4	d1

A->B	Yes	AB->C	No
B->A	Yes	AB->D	Yes
A->D	Yes	AD->B	Yes
D->A	No	CD->A	Yes
C->B	Yes	ABC->D	Yes
C->D	Yes	ABD->C	No

Functional Dependency Cont..:

- Functional Dependency is based on the concept of **Super Key**. It typically exists between the primary key and non-key attribute within a table.

Let we have a Department table with two attributes − **DeptId and DeptName**.

- Here, DeptId uniquely identifies the DeptName attribute. This is because if you want to know the department name, then at first you need to have the DeptId.

DeptId	DeptName
001	Finance
002	Marketing
003	HR

- Therefore, the above functional dependency between DeptId and DeptName can be determined as DeptName is functionally dependent on DeptId −

DeptId -> DeptName

Properties/Rules of Functional Dependency:

- **Reflexivity Rule (A1)** :

  if β is a subset of α then α -> β holds i.e. AB ->B

- **Augmentation Rule (A2)** :

  if α -> β holds then γ α -> γ β will also hold.

- **Transitivity Rule (A3)** :

  if α -> β and β-> γ holds then α -> γ will also hold.

- **Union Rule** :

  if α -> β and α -> γ holds then α -> β γ will also hold.

- **Decomposition Rule** :

  if α -> β γ holds then and α -> β & α -> γ will also hold.

Types of Functional Dependency:

- **Trivial functional dependency**:

  A ->B is trivial functional dependency if B is a subset of A.
  The following dependencies are trivial: A->A, AC->A

  For example:
  Consider a table with columns Student_id and Student_Name.
  {Student_Id, Student_Name} -> Student_Id is trivial FD.

- **Non trivial functional dependency**:

  If a functional dependency X->Y holds true where Y is not a subset of X then this dependency is called non trivial Functional dependency.

  For example:
  An employee table with attributes: emp_id, emp_name, emp_address.
  The following functional dependencies are non-trivial:
  emp_id -> emp_name (emp_name is not a subset of emp_id)
  emp_id -> emp_address (emp_address is not a subset of emp_id)

Closure of Functional Dependency:

- The Closure of Functional Dependency means the complete set of all possible attributes that can be functionally derived from given functional dependency using the inference rules known as Armstrong's Rules.

- If "F" is a functional dependency then closure of functional dependency can be denoted using "$\{F\}^{+}$".

There are three steps to calculate closure of functional dependency. These are:

- **Step-1** : Add the attributes which are present on Left Hand Side in the original functional dependency.
- **Step-2** : Now, add the attributes present on the Right Hand Side of the functional dependency.
- **Step-3** : With the help of attributes present on Right Hand Side, check the other attributes that can be derived from the other given functional dependencies. Repeat this process until all the possible attributes which can be derived are added in the closure.

Closure of Functional Dependency: Example

Example-1 : Consider the table student_details having (Roll_No, Name,Marks, Location) as the attributes and having two functional dependencies.

FD1 : Roll_No -> Name, Marks

FD2 : Name -> Marks, Location

{Roll_no}$^+$ = {Roll_No, Marks, Name, Location}

{Name}$^+$ = {Name, Marks, Location}

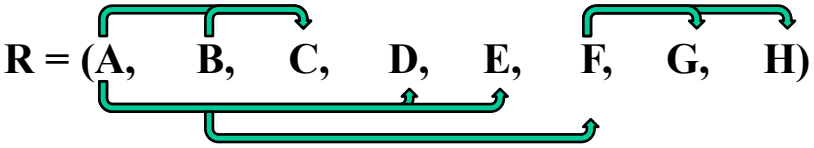
{Marks}$^+$ = {Marks}

{Location}$^+$ ={Location}

Determination of candidate keys in a relation:

Example 1: Consider a relation schema R = (A, B, C, D, E, F, G, H) on which the following functional dependencies hold: {AB–>C, A-> DE, B–>F, F–>GH}. Find how many candidate keys are possible in R?

Solution: Step-1: First draw above dependencies in edges diagram-



Step-2: Now find the attributes that don't have any incoming edge i.e. A & B.

It means, no other attributes can find A & B. So these are essential attributes and definitely will be part of all possible candidate keys.

Step-3: Now find the closure of essential attributes first.

$\{AB\}^+ = \{A, B, C, D, E, F, G, H\}$

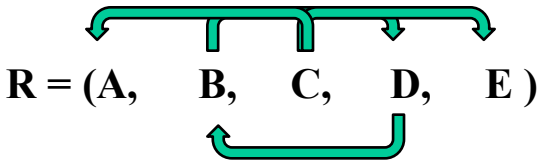
Step-4: Since $\{AB\}^+$ contains all the attributes of R, so it will acts as candidate key.

If essential attributes (i.e. AB) itself is a candidate key then no other combination need to check for candidate keys.

Determination of candidate keys in a relation:

Example 2: Consider a relation schema R = (A, B, C, D, E) on which the following functional dependencies hold: {CB–>ADE, D-> B}. Find no. of candidate keys in R?

Solution: Step-1: First draw above dependencies in edges diagram-



Step-2: Now find the attributes that don't have any incoming edge i.e. C.

Step-3: Now find the closure of essential attributes first.

$\{C\}^+ = \{ C \}$

Step-4: Since $\{C\}^+$ can`t access R. So we have to make combinations with C.

$\{CA\}^+ = \{ C, A \}$

$\{CB\}^+ = \{ C, B, A, D, E \}$

$\{CD\}^+ = \{ C, D, B, A, E \}$

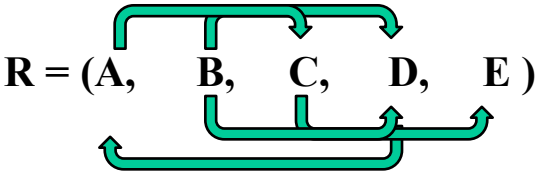
$\{CE\}^+ = \{ C, E \}$

Since $\{CB\}^+$ & $\{CD\}^+$ contains all attributes of R, so possible candidate keys are CB & CD for given relation R.

Determination of candidate keys in a relation:

Example 3: Consider a relation schema R = (A, B, C, D, E) on which the following functional dependencies hold: {AB->CD, D->A, CB–>DE}. Find no. of candidate keys in R?

Solution: Step-1: First draw above dependencies in edges diagram-



Step-2: Now find the attributes that don't have any incoming edge i.e. B.

Step-3: Now find the closure of essential attributes first.

$\{B\}^+ = \{ B \}$

Step-4: Since $\{B\}^+$ can`t access R. So we have to make combinations with B.

$\{BA\}^+ = \{ A, B, C, D, E\}$

$\{BC\}^+ = \{ B, C, D, E, A \}$

$\{BD\}^+ = \{ B, D, A, C, E \}$

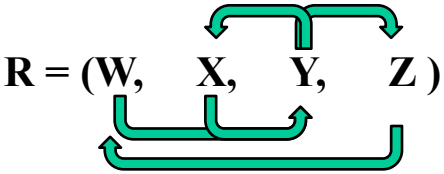
$\{BE\}^+ = \{ B, E \}$

Possible candidate keys are AB, BC & BD for given relation R.

Determination of candidate keys in a relation:

Example 4: Consider a relation schema R = (W, X, Y, Z) on which the following functional dependencies hold: {Z->W, Y->XZ, WX->Y}. Find no. of candidate keys in R?

Solution: Step-1: First draw above dependencies in edges diagram-



Step-2: All attributes are having incoming edge. So check all combinations.

Step-3: Now find the closure of all attributes.

$\{W\}^+ = \{ W \}$

$\{X \}^+ = \{ X \}$

$\{Y \}^+ = \{ Y, X, Z, W \}$

$\{Z \}^+ = \{ Z \}$

Here Y is candidate key. Also check combinations of W, X & Z

$\{WX \}^+ = \{ W, X, Y, Z \}$

$\{WZ \}^+ = \{ W, Z \}$

$\{XZ \}^+ = \{ X, Z, W, Y \}$

Thus possible candidate keys in R- {Y, WX, XZ}

Determination of candidate keys in a relation:

Example 5: Find the no of candidate keys for given dependencies:

i)

AB-> C
DC-> AE
E-> F

Keys- {ABD, BCD}

ii)

AB-> C
BD-> EF
AD- GH
A-> I

Keys- {ABD}

iii)

AB-> CD
D-> A
BC-> DE

Keys- {AB, BD, BC}

Canonical/Minimal Cover of Functional Dependency:

- In any relational model, there exists a set of functional dependencies. These functional dependencies when closely observed might contain redundant attributes.

- The ability of removing these redundant attributes without affecting the capabilities of the functional dependency is known as "**canonical cover of functional dependency**".

- Canonical cover of functional dependency is sometimes also referred to as "**minimal cover**".

- Canonical cover of functional dependency is denoted using "Fc".

- There are three steps to calculate the canonical cover for a relational schema having set of functional dependencies.

Canonical/Minimal Cover of Functional Dependency: Example

Example-1 Find the minimal cover for given functional dependency:

- **FD1 : A -> B**
- **FD2 : B -> C**
- **FD3 : A -> C**

Solution:

In above dependencies, FD3 (i.e. A->C) is redundant because it can be derived from FD1 & FD2 using transitivity rule.

So minimal cover will be

Fc= { **A -> B**
B -> C
}

Canonical/Minimal Cover of Functional Dependency: Example

Example 2: Consider a relation R(A,B,C,D) having some attributes and below are mentioned functional dependencies.

- **FD1 : B -> A**
- **FD2 : AD -> C**
- **FD3 : C -> ABD**

Step-1 : Decompose the functional dependencies using Decomposition rule i.e. single attribute on right hand side.

FD1 : B -> A
FD2 : AD -> C
FD3 : C -> A
FD4 : C -> B
FD5 : C -> D

Step-2 : Remove extraneous attributes from LHS of functional dependencies by calculating the closure of FD's having two or more attributes on LHS.

Here, only one FD has two or more attributes of LHS i.e. AD -> C.

$\{A\}^+ = \{A\}$
$\{D\}^+ = \{D\}$

In this case, attribute "A" can only determine "A" and "D" can only determine "D".
Hence, no extraneous attributes are present and the FD will remain the same and will not be removed.

Canonical/Minimal Cover of Functional Dependency: Example

FD1 : B -> A

FD2 : AD -> C

FD3 : C -> A

FD4 : C -> B

FD5 : C -> D

Step-3 : Remove FD's having transitivity.

Above FD1, FD3 and FD4 are forming transitive pair. Therefore we will have the following FD's left :

FD1 : B -> A

FD2 : AD -> C

FD3 : C -> B

FD4 : C -> D

Hence, the canonical cover of the relation R(A,B,C,D) will be:

Fc {R(ABCD)} = {B -> A , AD -> C, C -> BD}

Canonical/Minimal Cover of Functional Dependency: Example

Example 3: Consider the following set F of functional dependencies:

F= { **A -> BC**
B -> C
A -> B
AB -> C
}

Steps to find canonical cover:

1. There are two functional dependencies with the same set of attributes on the left:

A -> BC

A -> B

These two can be combined to get

A -> BC.

Now, the revised set F becomes:

F= { **A -> BC**
B -> C
AB -> C
}

2. There is an extraneous attribute in AB -> C because even after removing AB -> C from the set F, we get the same closures. This is because B -> C is already a part of F.

Now, the revised set F becomes:

F= { **A -> BC**
B -> C
}

3. C is an extraneous attribute in A -> BC, also A -> B is logically implied by A -> B and B -> C (by transitivity).

F= { **A -> B**
B -> C
}

4. After this step, F does not change anymore. So, Hence the required canonical cover is,

$F_{\{c\}}$= { **A -> B**
B -> C
}

Canonical/Minimal Cover of Functional Dependency: Example

Example-4 Consider a relation R (A,B,C,D,E,H). Find the minimal cover for given functional dependency:

- **FD1 : A -> C**
- **FD2 : AC -> D**
- **FD3 : E -> ADH**

Step-1 : Decompose the functional dependencies.

FD1 : A -> C
FD2 : AC -> D
FD3 : E -> A
FD4 : E -> D
FD5 : E -> H

Step-2 : Remove extraneous attributes from LHS of functional dependencies by calculating the closure of FD's having two or more attributes on LHS i.e. AC -> D.

$\{A\}^+ = \{A\}$
$\{C\}^+ = \{C\}$

In this case, attribute "A" can only determine "A" and "C" can only determine "C".
Hence, no extraneous attributes are present and the FD will remain the same and will not be removed.

FD1 : A -> C
FD2 : AC -> D
FD3 : E -> A
FD4 : E -> D
FD5 : E -> H

Step-3 : Remove FD's having transitivity.

There is no transitive dependency in above dependencies. So dependency will remove.

Hence, the canonical cover of the given relation will be:

Fc { **FD1 : A -> C**
FD2 : AC -> D
FD3 : E -> ADH
}

Canonical/Minimal Cover of Functional Dependency: Example

Example-5 Find the minimal cover of the set of functional dependencies given;
{A → C, AB → C, C → DI, CD → I, EC → AB, EI → C}

1. Right Hand Side (RHS) of all FDs should be single attribute.

F1 = {A → C, AB → C, C → D, C → I, CD → I, EC → A, EC → B, EI → C }

2. Remove extraneous attributes.

In the set of FDs, AB → C, CD → I, EC → A, EC → B, and EI → C have more than one attribute in the LHS. Find the closure of each attribute on the LHS

(i) A+ = ACDI

(ii) B+ = B

(iii) C+ = CDI

(iv) D+ = D

(v) E+ = E

(vi) I+ = I

From (i), the closure of A included the attribute C. So, B is extraneous in AB → C, and B can be removed.

From (iii), the closure of C included the attribute I. So, D is extraneous in CD → I, and D can be removed.

No more extraneous attributes are found. Hence, we write F1 as F2 after removing extraneous attributes from F1 as follows;

F2 = {A → C, C → D, C → I, EC → A, EC → B, EI → C}

Canonical/Minimal Cover of Functional Dependency: Example

3. Eliminate redundant functional dependency using transitivity.

None of the FDs in F2 is redundant. Hence, F2 is minimal cover.

Hence, set of functional dependencies F2 is the minimal cover for the set F.

Fc = { **A → C,**
C → D,
C → I,
EC → A,
EC → B,
EI → C
}

OR

Fc= { A →C, C →DI, EC →AB, EI →C}

Closure of Functional Dependency: Calculating Candidate Key

"A Candidate Key of a relation is an attribute or set of attributes that can determine the whole relation or contains all the attributes in its closure."

Example-1 : Consider the relation R(A,B,C) with given functional dependencies :

FD1 : A-> B

FD2 : B-> C

- $\{A\}^+ = \{A, B, C\}$
- $\{B\}^+ = \{B, C\}$
- $\{C\}^+ = \{C\}$

Clearly, "A" is the candidate key as, its closure contains all the attributes present in the relation "R".

Closure of Functional Dependency: Calculating Candidate Key

Example-2 : Consider another relation R(A, B, C, D, E) having the FDs :

FD : A -> BC, C -> B, D -> E, E -> D

- $\{A\}^+ = \{A, B, C\}$
- $\{B\}^+ = \{B\}$
- $\{C\}^+ = \{C, B\}$
- $\{D\}^+ = \{E, D\}$
- $\{E\}^+ = \{E, D\}$

In this case, a single attribute is unable to determine all the attribute on its own. Here, we need to combine two or more attributes to determine the candidate keys.

- $\{A, D\}^+ = \{A, B, C, D, E\}$
- $\{A, E\}^+ = \{A, B, C, D, E\}$

Hence, "AD" and "AE" are the two possible keys of the given relation “R”.

Closure of Functional Dependency: Calculating Candidate Key

GATE Question 1:

Consider the relation scheme R = {E, F, G, H, I, J, K, L, M, N} and the set of functional dependencies {{E, F} -> {G}, {F} -> {I, J}, {E, H} -> {K, L}, K -> {M}, L -> {N} on R. What is the key for R?

A. {E, F} **B. {E, F, H}**

C. {E, F, H, K, L} **D. {E}**

Answer: Finding attribute closure of all given options, we get:

$\{E,F\}^+ = \{EFGIJ\}$

$\{E,F,H\}^+ = \{EFHGIJKLMN\}$

$\{E,F,H,K,L\}^+ = \{\{EFHGIJKLMN\}$

$\{E\}^+ = \{E\}$

$\{EFH\}^+$ and $\{EFHKL\}^+$ results in set of all attributes, but EFH is minimal. So it will be candidate key. So correct option is (B).

Closure of Functional Dependency: Calculating Candidate Key

GATE Question 2:

In a schema with attributes A, B, C, D and E following set of functional dependencies are given

{A -> B, A -> C, CD -> E, B -> D, E -> A}

Which of the following functional dependencies is NOT implied by the above set?

A. CD -> AC

B. BD -> CD

C. BC -> CD

D. AC -> BC

Answer: Using FD set given in question,

(CD)+ = {CDEAB} which means CD -> AC also holds true.

(BD)+ = {BD} which means BD -> CD can't hold true.

So this FD is no implied in FD set. So (B) is the required option.

Others can be checked in the same way.

Closure of Functional Dependency: Calculating Candidate Key

GATE Question 3:

Consider a relation scheme R = (A, B, C, D, E, H) on which the following functional dependencies hold: {A–>B, BC–> D, E–>C, D–>A}. What are the candidate keys of R?

(a) AE, BE

(b) AE, BE, DE

(c) AEH, BEH, BCH

(d) AEH, BEH, DEH

Answer: (AE)+ = {ABECD} which is not set of all attributes. So AE is not a candidate key. Hence option A and B are wrong.

(AEH)+ = {ABCDEH}

(BEH)+ = {BEHCDA}

(BCH)+ = {BCHDA}

which is not set of all attributes. So BCH is not a candidate key. Hence option C is wrong.

So correct answer is D.

What is Decomposition?

- Decomposition of a relation is done when a relation in relational model is not in appropriate normal form.
- Decomposition is the process of breaking a relation into multiple relations.
- Two types of Decomposition- **Lossless Join & Lossy Join.**
- *A decomposition is said to be 'Lossless' if it confirms that the information in the original relation can be accurately reconstructed based on the decomposed relations.*
- *If there is no proper decomposition of the relation, then it may lead to problems like loss of information and it is called 'Lossy Decomposition'*
- If a relation R is decomposed into two or more relations then it must be lossless join as well as dependency preserving.

Properties of Decomposition

1. **Lossless Decomposition**

2. **Dependency Preservation**

3. **Lack of Data Redundancy**

Lossless Join Decomposition

- Decomposition must be lossless. It means that the information should not get lost from the relation that is decomposed.
- It gives a guarantee that the join will result in the same relation as it was decomposed.
- Consider there is a relation R which is decomposed into sub relations R1 , R2 , …. , Rn.
- This decomposition is called lossless join decomposition when the join of the sub relations results in the same relation R that was decomposed.
- For lossless join decomposition, we always have-

$$R_1 \bowtie R_2 \bowtie R_3 ..... \bowtie R_n = R$$

where $\bowtie$ is a natural join operator

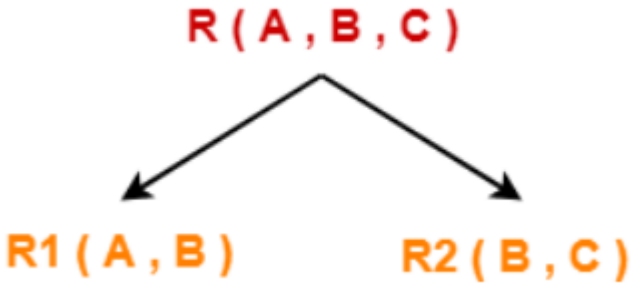
Lossless Join Decomposition Example 1

Consider the following relation R(A , B , C)-

A	B	C
1	2	1
2	5	3
3	3	3

R(A , B , C)

Consider this relation is decomposed into two sub relations R1(A ,B) & R2(B,C)-



A	B
1	2
2	5
3	3

R_1(A , B)

B	C
2	1
5	3
3	3

R_2(B , C)

Lossless Join Decomposition Example 1 Cont….

Now, let us check whether this decomposition is lossless or not.

For lossless decomposition, we must have-

$$R1 \bowtie R2 = R$$

Now, if we perform the natural join ($\bowtie$) of the sub relations R1 and R2 , we get-

A	B
1	2
2	5
3	3

$R_1(A, B)$

$\bowtie$

B	C
2	1
5	3
3	3

$R_2(B, C)$

=

A	B	C
1	2	1
2	5	3
3	3	3

This relation is same as the original relation R.

Thus, we conclude that above decomposition is lossless join decomposition.

Lossy Decomposition

- As the name suggests, when a relation is decomposed into two or more relational schemas, the loss of information is unavoidable when the original relation is retrieved.
- Consider there is a relation R which is decomposed into sub relations R1 , R2 , …. , Rn.
- This decomposition is called lossy join decomposition when the join of the sub relations does not result in the same relation R that was decomposed.
- The natural join of the sub relations is always found to have some extraneous tuples.
- For lossy join decomposition, we always have-

$$R_1 \bowtie R_2 \bowtie R_3 \ldots\ldots \bowtie R_n \supset R$$

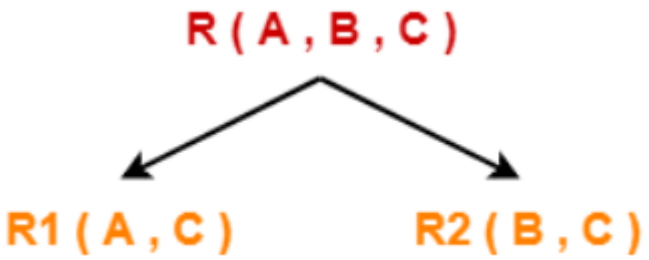
Lossy Decomposition Example-1

Consider the following relation R(A , B , C)-

A	B	C
1	2	1
2	5	3
3	3	3

R(A , B , C)

Consider this relation is decomposed into two sub relations as R1(A , C) and R2(B , C)-



A	C
1	1
2	3
3	3

R_1(A , B)

B	C
2	1
5	3
3	3

R_2(B , C)

Lossy Decomposition Example-1 Cont….

Now, let us check whether this decomposition is lossy or not.

- For lossy decomposition, we must have-

  R1 ⋈ R2 ⊃ R

- Now, if we perform the natural join (⋈) of the sub relations R1 and R2 we get-

A	C
1	1
2	3
3	3

$R_1(A, B)$

⋈

B	C
2	1
5	3
3	3

$R_2(B, C)$

=

A	B	C
1	2	1
2	5	3
2	3	3
3	5	3
3	3	3

This relation is not same as the original relation R and contains some extraneous tuples. **Clearly, R1 ⋈ R2 ⊃ R.**

Thus, we conclude that the above decomposition is lossy join decomposition.

Lossless Join Decomposition Example 2

Let us see an example −

<EmpInfo>

Emp_ID	Emp_Name	Emp_Age	Emp_Location	Dept_ID	Dept_Name
E001	Jacob	29	Alabama	Dpt1	Operations
E002	Henry	32	Alabama	Dpt2	HR
E003	Tom	22	Texas	Dpt3	Finance

Decompose the above table into two tables:

<EmpDetails>

Emp_ID	Emp_Name	Emp_Age	Emp_Location
E001	Jacob	29	Alabama
E002	Henry	32	Alabama
E003	Tom	22	Texas

<DeptDetails>

Dept_ID	Emp_ID	Dept_Name
Dpt1	E001	Operations
Dpt2	E002	HR
Dpt3	E003	Finance

Lossless Join Decomposition Example 2 Cont….

Now, Natural Join is applied on the above two tables −

The result will be −

Emp_ID	Emp_Name	Emp_Age	Emp_Location	Dept_ID	Dept_Name
E001	Jacob	29	Alabama	Dpt1	Operations
E002	Henry	32	Alabama	Dpt2	HR
E003	Tom	22	Texas	Dpt3	Finance

Since this is equivalent to original relation R, so it is lossless join decomposition.

Lossy Join Decomposition Example 2

Let us see an example −

<EmpInfo>

Emp_ID	Emp_Name	Emp_Age	Emp_Location	Dept_ID	Dept_Name
E001	Jacob	29	Alabama	Dpt1	Operations
E002	Henry	32	Alabama	Dpt2	HR
E003	Tom	22	Texas	Dpt3	Finance

Decompose the above table into two tables −

<EmpDetails>

Emp_ID	Emp_Name	Emp_Age	Emp_Location
E001	Jacob	29	Alabama
E002	Henry	32	Alabama
E003	Tom	22	Texas

<DeptDetails>

Dept_ID	Dept_Name
Dpt1	Operations
Dpt2	HR
Dpt3	Finance

- Now, you won't be able to join the above tables, since Emp_ID isn't part of the DeptDetails relation.

- **Therefore, the above relation has lossy decomposition.**

Check for Lossless Join Decomposition

To check for lossless join decomposition using FD set, following conditions must hold:

1. Union of Attributes of R1 and R2 must be equal to attribute of R. Each attribute of R must be either in R1 or in R2.

$$Att(R1) \cup Att(R2) = Att(R)$$

2. Intersection of Attributes of R1 and R2 must not be NULL.

$$Att(R1) \cap Att(R2) \neq \Phi$$

3. Common attribute must be a key for at least one relation (R1 or R2)

$$Att(R1) \cap Att(R2) \rightarrow \text{Super Key of R1 or R2}$$

$$R1 \cap R2 \rightarrow R1$$
$$\text{OR}$$
$$R1 \cap R2 \rightarrow R2$$

Check for Lossless Join Decomposition Example-1

Example-1: Consider a relation schema R (A , B , C , D) with the functional dependencies A → B and C → D. Determine whether the decomposition of R into R1 (A , B) and R2 (C , D) is lossless or lossy.

Solution:

To determine whether the decomposition is lossless or lossy, we will check all the conditions one by one. If any of the conditions fail, then the decomposition is lossy otherwise lossless.

Condition-01: (R1 U R2 = R)

$$R1 (A , B) \cup R2 (C , D) = R (A , B , C , D)$$

Clearly, union of the sub relations contain all the attributes of relation R. Thus, condition-01 satisfies.

Condition-02: ($R1 \cap R2 \neq \Phi$)

$$R1 (A , B) \cap R2 (C , D) = \Phi$$

Clearly, intersection of the sub relations is null. So, condition-02 fails.

Thus, we conclude that the decomposition is lossy.

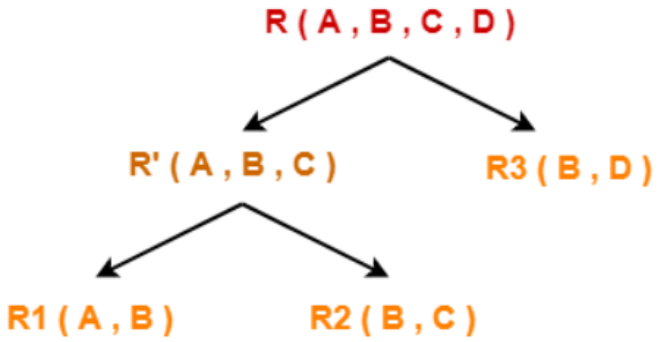
Check for Lossless Join Decomposition Example-2

Example-2: Consider a relation schema R (A , B , C , D) with the following functional dependencies- {A → B, B → C, C → D, D → B}. Determine whether the decomposition of R into R1 (A , B) , R2 (B , C) and R3 (B , D) is lossless or lossy.

Solution:

When a given relation is decomposed into more than two sub relations, then-

- First, divide the given relation into two sub relations.
- Then, divide the sub relations according to the sub relations given in the question.



To determine whether the decomposition is lossless or lossy, we will check all the conditions one by one.
If any of the conditions fail, then the decomposition is lossy otherwise lossless.

Check for Lossless Join Decomposition Example-2 Cont...

Condition-01: (R1 U R2 = R) R‘ (A , B , C) ∪ R3 (B , D) = R (A , B , C , D)

Clearly, union of the sub relations contain all the attributes of relation R. Thus, condition-01 satisfies.

Condition-02: (R1 ∩ R2 ≠ Φ) R‘ (A , B , C) ∩ R3 (B , D) = B

Clearly, intersection of the sub relations is not null. Thus, condition-02 satisfies.

Condition-03: (R1 ∩ R2 → R1 OR R1 ∩ R2 → R2)

R‘ (A , B , C) ∩ R3 (B , D) = B

Now, the closure of attribute B is- *Given FDs: {A → B, B → C, C → D, D → B}*

B+ = { B , C , D }

Clearly, intersection of the sub relations is a super key of one of the sub relations. So, condition-03 satisfies.

Thus, we conclude that the decomposition R` & R3 is lossless.

Check for Lossless Join Decomposition Example-2 Cont...

Decomposition of R'(A, B, C) into R1(A, B) and R2(B, C)-

Condition-01: (R1 U R2 = R) **R1 (A , B) ∪ R2 (B , C) = R' (A , B , C)**

Thus, condition-01 satisfies.

Condition-02: (R1 ∩ R2 ≠ Φ) **R1 (A , B) ∩ R2 (B , C) = B**

Thus, condition-02 satisfies.

Condition-03: (R1 ∩ R2 → R1 OR R1 ∩ R2 → R2)

R1 (A , B) ∩ R2 (B , C) = B

Now, the closure of attribute B is- *Given FDs: {A → B, B → C, C → D, D → B}*

B+ = { B , C , D }

Attribute 'B' is a super key of the sub relation R2 thus *decomposition is lossless.*

Conclusion- **Overall decomposition of relation R into sub relations R1, R2 and R3 is Lossless.**

Check for Lossless Join Decomposition Example

Example-3 Consider a relation R(A,B,C,D,E) and functional dependencies are F={AC→B,C→D,A→E,C→B}. R is decomposed into R1(A,B,C) and R2(C,D) Then is it a lossless decomposition?

Solution: **Lossy Join Decomposition**

Example-4 Check for the given relations, whether they are lossless or not-

R(A,B,C,D,E)

FD = {A->BC, CD->E. B->D, E->A}

R1(A,B,C) & R2 (A,D,E)

Lossless Join Decomposition

Since, R1 ∩ R2=A (A is key in R1)

Check for Lossless Join Decomposition Example

Example-4 Check for the given relations, whether they are lossless or not-

i)

R(A,B,C)

FD = {A->B}

R1(A,B) & R2 (B,C)

Lossy Join Decomposition

Since, R1 ∩ R2=B (B is not key in either R1 or R2)

ii)

R(A,B,C)

FD = {A->B}

R1(A,B) & R2 (A,C)

Lossless Join Decomposition

Since, R1 ∩ R2=A (A is key in R1)

iii)

R(A,B,C,D)

FD = {A->B, A->C, C->D}

R1(A,B,C) & R2 (C,D)

Lossless Join Decomposition

Since, R1 ∩ R2=C (C is key in R2)

Dependency Preservation:

- Dependency preservation is another property of decomposed relations
- A decomposition of a relation R into R_1, R_2, R_3, …, R_n is dependency preserving only if the following is hold;

$$(F_1 \cup F_2 \cup F_3 \cup \ldots \cup F_n)^+ = F^+$$

where, F_1, F_2, F_3,… F_n -set of Functional dependencies of relations R_1, R_2, R_3, …, R_n respectively.

If the closure of set $(F_1 \cup F_2 \cup F_3 \cup \ldots \cup F_n)^+$ are equal to the set of functional dependencies of the main relation R (before decomposition), then we would say the decomposition is lossless dependency preserving.

Dependency Preservation Example 1:

Example 1:

Assume R(A, B, C, D) with FDs A→B, B→C, C→D.

Let us decompose R into R1 and R2 as follows;

R1(A, B, C)

R2(C, D)

Then find decomposition is dependency preserving or not?

Solution:

The FDs A→B, and B→C are hold in R1.

The FD C→D holds in R2.

Since, all the functional dependencies hold here. Hence, this decomposition is dependency preserving.

Dependency Preservation Example 2:

Example 2:

Let, R (X, Y, Z) is decomposed into R1 (X, Y) & R2 (Y, Z)

& given set of FDs= {X->Y, Y->Z, Z->X}

Check whether decomposition is

i) Lossless or Lossy

ii) dependency preserving or not

Solution:

i) Check for Lossless decomposition:

R1 U R2 ={X, Y, Z} *//Condition-1 is Satisfied*

R1 ∩ R2 ≠ Φ *//Condition-2 is Satisfied*

R1 ∩ R2 = {Y}

$\{Y\}^{+}$= {Y, Z, X}

Thus, Y is key in R2 *//Condition-3 is Satisfied*

So given decomposition is Lossless Join Decomposition.

Dependency Preservation Example 2 Cont.…:

Given, FDs= {X->Y, Y->Z, Z->X} & R1 (X, Y) & R2 (Y, Z)

ii) Now check for dependency preservation:

For R1, find closure of X & Y

X^+ ={X, Y, Z}
={X, Y} //since Z is not in R1
={Y} //since X is trivial attribute

So FD is: X->Y

Y+ ={Y, Z, X}
= {X, Y} //since Z is not in R1
= {X} //since Y is trivial attribute

So FD is: Y->X

For R2, find closure of Y & Z

Y^+ ={Y, Z, X}
={Y, Z} //since X is not in R2
={Z} //since Y is trivial attribute

So FD is: Y->Z

Z+ ={Z, X, Y}
= {Z, Y} //since X is not in R2
= {Y} //since Z is trivial attribute

So FD is: Z->Y

Dependency Preservation Example 2 Cont.…:

Given, FDs= {X->Y, Y->Z, Z->X} & R1 (X, Y) & R2 (Y, Z)

So, we have

FD1= {X->Y, Y->X} & FD2= {Y->Z, Z->Y}

Now check for the given FDs= {X->Y, Y->Z, Z->X}

i) X->Y is present in FD1

ii) Y-> Z is present in FD2

iii) Z-> X can be inferred from FD1 & FD2 using transitive dependency between Z->Y & Y-> X

So given decomposition is dependency preserving

Dependency Preservation Example 3:

Example 3:

Consider a relation R (P, Q, R, S) with a set of Functional Dependency

FD = {PQ→R, R→S, S→P}

Relation R is decomposed into R1 (P, Q, R) and R2(R, S).

Find whether the decomposition is dependency preserving or not.

Solution:

To solve this problem, we need to find the closure of Functional Dependencies FD1 and FD2 of the relations R1 (P, Q, R) and R2(R, S).

1) To find the closure of FD1, we have to consider all combinations of (P, Q, R). i.e., we need to find out the closure of P, Q, R, PQ, QR, and RP.

closure (P) = {P} // Trivial

closure (Q) = {Q} // Trivial

closure (R) = {R, P, S}

= {R, P} //but S can't be in closure as S is not present in R1.

= {P} // Removing R from right side as it is trivial attribute

So FD is: R-> P

Dependency Preservation Example 3 Cont.…:

Given, FD = {PQ→R, R→S, S→P} & R1 (P, Q, R) and R2 (R, S).

closure (PQ) = *{P, Q, R, S}*

= *{P, Q, R} // since S is not in R1*

= *{R} // Removing PQ from right side as these are trivial*

So FD is : PQ -> R

closure (QR) = *{Q, R, S, P}*

= *{P, Q, R} // since S is not in R1*

= *{P} // Removing QR from right side as these are trivial*

So FD is: QR -> P

Closure (PR) = *{P, R, S}*

= *{P,R} // since S is not in R1*

= *{ } // Removing PR from right side as these are trivial*

FD1 {R -> P, PQ -> R, QR -> P}.

Dependency Preservation Example 3 Cont.…:

Given, FD = {PQ→R, R→S, S→P} & R1 (P, Q, R) and R2 (R, S).

FD1 {R -> P, PQ -> R, QR -> P}.

2) Similarly FD2 {R-> S}

In the original Relation Dependency FD= {PQ→R, R→S, S→P}.

PQ --> R is present in FD1.

R --> S is present in FD2.

S --> P is not preserved.

So as a consequence, given decomposition is not dependency preserving.

Normalization:

- Normalization is the process of organizing the data in the database.
- Normalization is used to minimize the redundancy from a relation or set of relations.
- It is also used to eliminate the undesirable characteristics like Insertion, Update and Deletion Anomalies.
- Normalization divides the larger table into the smaller table and links them using relationship.
- Let’s discuss about anomalies first then we will discuss normal forms with examples.

Anomalies in DBMS

- There are three types of anomalies that occur when the database is not normalized. These are – Insertion, update and deletion anomaly.

Example: Suppose a manufacturing company stores the employee details in a table named employee that has four attributes:

Emp_id	Emp_name	Emp_address	Emp_dept
101	Raman	Delhi	D001
101	Raman	Delhi	D002
123	Mayur	Agra	D890
166	Gagan	Chennai	D900
166	Gagan	Chennai	D004

The above table is not normalized. We will see the problems that we face when a table is not normalized.

Anomalies in DBMS Cont..

i. Update Anomaly:

Emp_id	Emp_name	Emp_address	Emp_dept
101	Raman	Delhi	D001
101	Raman	Delhi	D002
123	Mayur	Agra	D890
166	Gagan	Chennai	D900
166	Gagan	Chennai	D004

- In the above table we have two rows for employee Raman as he belongs to two departments of the company.
- If we want to update the address of Raman then we have to update the same in two rows or the data will become inconsistent.

Anomalies in DBMS Cont..

ii. Insert anomaly:

Emp_id	Emp_name	Emp_address	Emp_dept
101	Raman	Delhi	D001
101	Raman	Delhi	D002
123	Mayur	Agra	D890
166	Gagan	Chennai	D900
166	Gagan	Chennai	D004

- Suppose a new employee joins the company, who is under training and currently not assigned to any department.
- Then we would not be able to insert the data into the table if Emp_dept field doesn't allow Nulls.

Anomalies in DBMS Cont..

iii. Delete anomaly:

Emp_id	Emp_name	Emp_address	Emp_dept
101	Raman	Delhi	D001
101	Raman	Delhi	D002
123	Mayur	Agra	D890
166	Gagan	Chennai	D900
166	Gagan	Chennai	D004

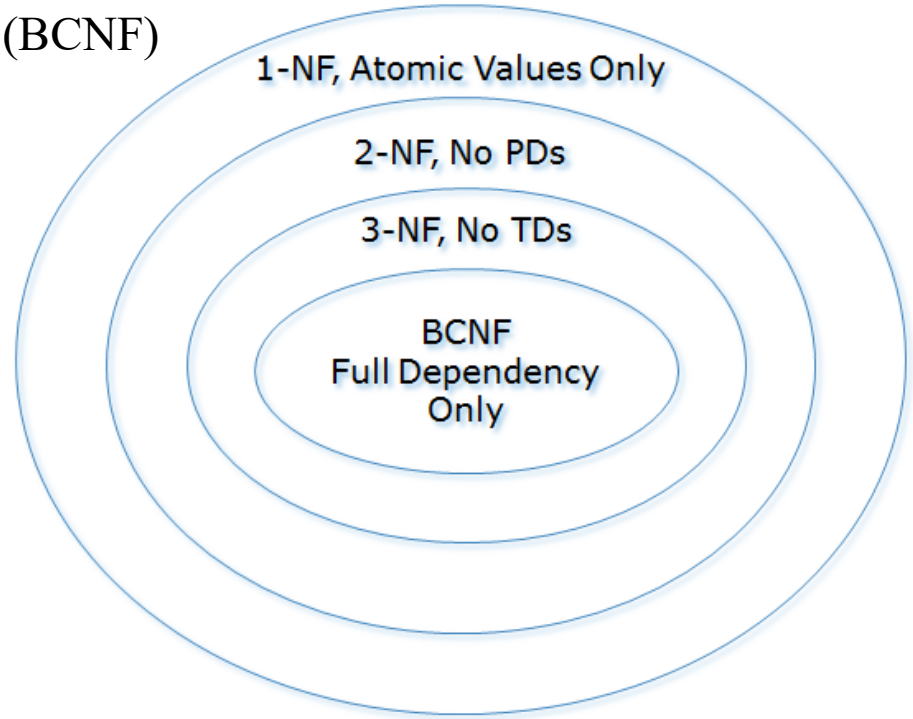
- Suppose, if at a point of time, the company closes the department D890.
- Then deleting the rows that are having emp_dept as D890 would also delete the information of employee Mayur since he is assigned only to this department.

To overcome these anomalies we need to normalize the data.

Normalization:

Here are the most commonly used normal forms:

- First Normal Form (1NF)
- Second Normal Form (2NF)
- Third Normal Form (3NF)
- Boyce & Codd Normal Form (BCNF)
- Fourth Normal Form (4NF)



*PD: Partial Dependencies
*TD: Transitive Dependencies

1st Normal Form:

According to I Normal Form:

" A relation is said to be in first normal form, if every attribute in that relation is single (atomic) valued attribute."

- *If a relation contains composite or multi-valued attribute, it violates first normal form.*

For a table to be in the First Normal Form, it should follow the following rules:

- It should only have single (atomic) valued attributes/columns.
- Values stored in a column should be of the same domain

1st Normal Form Example:

Suppose a company wants to store the names and contact details of its employees. It creates a table that looks like this:

Emp_Id	Emp_Name	Emp_Address	Emp_Mobile
101	Harsh	New Delhi	8912312390
102	Jay	Kanpur	8812121212 9900012222
103	Ravi	Chennai	7778881212
104	Lokesh	Bangalore	9990000123 8123450987

- Two employees (Jay & Lokesh) are having two mobile numbers so the company stored them in the same field as you can see in the table above.

- This table is not in 1NF as the rule says “each attribute of a table must have atomic (single) values”, the emp_mobile values for employees Jay & Lokesh violates that rule.

1st Normal Form Example Cont..:

Solution:1 Insert records for each mobile number-

emp_id	emp_name	emp_address	emp_mobile
101	Harsh	New Delhi	8912312390
102	Jay	Kanpur	8812121212
102	Jay	Kanpur	9900012222
103	Ravi	Chennai	7778881212
104	Lokesh	Bangalore	9990000123
104	Lokesh	Bangalore	8123450987

Solution:2 Add number of columns for each mobile number-

emp_id	emp_name	emp_address	emp_mobile1	emp_mobile1
101	Harsh	New Delhi	8912312390	8912312390
102	Jay	Kanpur	8812121212	9900012222
103	Ravi	Chennai	7778881212	7778881212
104	Lokesh	Bangalore	9990000123	8123450987

1st Normal Form Example Cont..:

Solution:3 Decompose the relation-

emp_id	emp_name	emp_address
101	Harsh	New Delhi
102	Jay	Kanpur
103	Ravi	Chennai
104	Lokesh	Bangalore

emp_details

emp_id	emp_mobile
101	8912312390
102	8812121212
102	9900012222
103	7778881212
104	9990000123
104	8123450987

emp_contact

2nd Normal Form:

A relation is said to be in 2NF –

When it is in 1NF and every non prime attributes should be fully functional dependent on the key of R.

Or

A relation is said to be in 2NF-

When it is in 1NF & it doesn't contain any partial dependency

- **Prime Attribute:** An attribute that is part of candidate key.
- **Non-Prime Attribute:** An attribute that is not part of any candidate key.

2nd Normal Form Cont..:

Fully Functional Dependency-

- An attribute is fully functional dependent on another attribute, if it is Functionally Dependent on that attribute and not on any of its proper subset.

- A functional dependency **α β -> γ** is said to be fully functional dependent, if and only if

α -> γ
and **β -> γ** both are **NOT** hold

- It means if **γ** can be accessed by combination of **α β** then it should not be functionally dependent on part of **α & β.**

Partial Dependency-

- Partial Dependency occurs when a nonprime attribute is functionally dependent on part of a candidate key.

- The 2nd Normal Form (2NF) eliminates the Partial Dependency.

2nd Normal Form Example:

Example: Let's assume, a relation-

TEACHER_ID	SUBJECT	TEACHER_AGE
25	Chemistry	30
25	Biology	30
47	English	35
83	Math	38
83	Chemistry	38
97	English	35

Candidate Key:
{TEACHER_ID, SUBJECT}

Non-prime attribute TEACHER_AGE is dependent on TEACHER_ID which is a proper subset of a candidate key.
That's why it violates the rule for 2NF.

TEACHER_DETAIL

TEACHER_ID	TEACHER_AGE
25	30
47	35
83	38
97	35

TEACHER_SUBJECT

TEACHER_ID	SUBJECT
25	Chemistry
25	Biology
47	English
83	Math
83	Chemistry
97	English

2nd Normal Form Example 1

Example-1:

Let a relation R (A,B,C,D,E) has following functional dependencies-

{ AB->C, B->D, A->E}. Normalize it up to 2NF.

Solution:

Check for 1NF:

- Relation is already in 1 Normal Form, since no records are given in tabular format so assume that relation R (A,B,C,D,E) is having all atomic values.

Check for 2NF:

- Find the candidate keys of R

  (AB)+= {A, B, C, D, E}

- **Prime attributes- {A, B}** //Part of candidate key
- **Non prime attributes- {C, D, E}**

2nd Normal Form Example 1 Cont..

Given, R (A,B,C,D,E) FDs={AB->C, B->D, A->E}

Now from the definition of 2NF, we have to check that every non prime attributes (i.e. C, D, E) should be fully functional dependent on key of R

- **Check for {C} – i.e. {AB->C}**
  - AB-> C is fully functional dependent if A-> C & B-> C don't hold.
  - Since there are no such dependencies present in the given FDs
  - **Thus, C is fully functional dependent on the candidate key of R**

- **Check for {D} – i.e. {AB->D}**
  - AB->D is fully functional dependent if A->D & B->D don't hold.
  - Since there is one dependency B->D is present in the given FDs.
  - **Thus, D is partially dependent on part of candidate key.**

- **Check for {E} – i.e. {AB->E}**
  - AB->E is fully functional dependent if A->E & B->E don't hold.
  - Since there is one dependency A->E is present in the given FDs.
  - **Thus, E is partially dependent on part of candidate key.**

2nd Normal Form Example 1 cont..

- **{C} – Fully functional dependent on key of R**
- **{D} – Partially dependent on key of R (B->D)**
- **{E} – Partially dependent on key of R (A->E)**

- Thus, D & E violates the definition of 2NF since both are partially dependent.
- In order to remove partial dependencies, we have to decompose relation R
- For the decompositions, we'll remove the attributes from R that are not fully functional dependent, i.e.

R1 (A, B, C)

- Put D & E in another relations, with the attribute upon which they hold partial dependencies, i.e.

R2 (B, D) *//for B->D*

R3 (A, E) *//for A->E*

- *So the given relation R (A,B,C,D,E) is decomposed in to*

R1(A, B, C)

R2 (B, D)

R3 (A, E)

And now they are in 2NF

2nd Normal Form Example 2

Example-2:

Let a relation R (A, B, C, D, E, F) has following functional dependencies-

{ A->BCDEF, BC->A } Check whether it is in 2NF or not.

Solution:

Check for 1NF:

- Relation is already in 1 Normal Form, since no records are given in tabular format so assume that relation R (A,B,C,D,E,F) is having all atomic values.

Check for 2NF:

- Find the candidate keys of R

(A)+= {A, B, C, D, E, F}

(BC)+= {B, C, A, D, E, F}

Candidate Keys= {A, BC}

- **Prime attributes- {A, B, C}** //Part of candidate key
- **Non prime attributes- {D, E. F}**

2nd Normal Form Example 2 Cont..

Given, R (A,B,C,D,E,F) FDs={A->BCDEF, BC->A}

Now from the definition of 2NF, we have to check that every non prime attributes (i.e. D, E, F) should be fully functional dependent on key of R

- **Check fully functional dependency on A**

  Since A is single attribute as candidate key, so from A, all the dependencies from would be fully functional dependent. (No need to check)

- **Check fully functional dependency on BC**

  BC->D //No partial dependency like B->D or C->D, so D is FFD on BC

  BC->E //No partial dependency like B->E or C->E, so E is FFD on BC

  BC->F //No partial dependency like B->F or C->F, so F is FFD on BC

Since, no any partial dependency is there, so given relation R is already in 2NF.

2nd Normal Form Example 3

Example-3:

Let a relation R (A, B, C, D, E, F) has following functional dependencies-

{ A->BCDEF, BC->A, B->F, C->E } Check whether it is in 2NF or not.

Solution:

Check for 1NF:

- Relation is already in 1 Normal Form, since no records are given in tabular format so assume that relation R (A,B,C,D,E,F) is having all atomic values.

Check for 2NF:

- Find the candidate keys of R

(A)+= {A, B, C, D, E, F}

(BC)+= {B, C, A, D, E, F}

Candidate Keys= {A, BC}

- **Prime attributes- {A, B, C}** //Part of candidate key
- **Non prime attributes- {D, E. F}**

2nd Normal Form Example 3 Cont..

Given, R (A,B,C,D,E,F) FDs={A->BCDEF, BC->A, B->F, C->E}

Now from the definition of 2NF, we have to check that every non prime attributes (i.e. D, E, F) should be fully functional dependent on key of R

- **Check fully functional dependency on A**

  Since A is single attribute as candidate key, so from A, all the dependencies from would be fully functional dependent. (No need to check)

- **Check fully functional dependency on BC**

  BC->D //No partial dependency like B->D or C->D, so D is FFD on BC

  BC->E //C->E is partial dependency, so E doesn't FFD on BC

  BC->F //B->F is partial dependency, so F doesn't FFD on BC

Given relation is not in 2NF, so decompose it-

R1(A, B, C, D)

R2(C, E)

R3(B, F)

3rd Normal Form:

A relation R is said to be in the Third Normal Form when,

- **It is in the Second Normal form and**
- **Every non prime attributes should not be transitive dependent on key of R**

Thus, 3NF violates transitive dependency.

Transitive dependency-

if **α -> β** and **β ->γ** hold,

then **α -> γ** will also holds

3rd Normal Form:

EMP_ID	EMP_NAME	EMP_ZIP	EMP_STATE	EMP_CITY
222	Harsh	201010	UP	Noida
333	Sanjay	02228	US	Boston
444	Pavan	60007	US	Chicago
555	Kartik	06389	UK	Norwich
666	Jay	462007	MP	Bhopal

Super key in the table above:

{EMP_ID}, {EMP_ID, EMP_NAME}, {EMP_ID, EMP_NAME, EMP_ZIP}....so on

Candidate key: {EMP_ID}

Non-prime attributes: In the given table, all attributes except EMP_ID are non-prime.

- Here, EMP_STATE & EMP_CITY dependent on EMP_ZIP and EMP_ZIP dependent on EMP_ID.
- The non-prime attributes (EMP_STATE, EMP_CITY) transitively dependent on super key (EMP_ID).
- It violates the rule of third normal form.

3rd Normal Form:

EMP_ID	EMP_NAME	EMP_ZIP	EMP_STATE	EMP_CITY
222	Harsh	201010	UP	Noida
333	Sanjay	02228	US	Boston
444	Pavan	60007	US	Chicago
555	Kartik	06389	UK	Norwich
666	Jay	462007	MP	Bhopal

That's why we need to move the EMP_CITY and EMP_STATE to the new <EMPLOYEE_ZIP> table, with EMP_ZIP as a Primary key.

EMP_ID	EMP_NAME	EMP_ZIP
222	Harsh	201010
333	Sanjay	02228
444	Pavan	60007
555	Kartik	06389
666	Jay	462007

EMP_ZIP	EMP_STATE	EMP_CITY
201010	UP	Noida
02228	US	Boston
60007	US	Chicago
06389	UK	Norwich
462007	MP	Bhopal

3rd Normal Form Example 1

Example-1:

Normalize a relation R (A, B, C, D, E) up to 3NF, when following functional dependencies are given { A->BCDE, C->D, B->E }

Solution:

Check for 1NF:

- Relation is already in 1 Normal Form, since no records are given in tabular format so assume that relation R (A,B,C,D,E) is having all atomic values.

Check for 2NF:

- Find the candidate keys of R

(A)+= {A, B, C, D, E} **Candidate Key= {A}**

- **Prime attributes- {A}** //Part of candidate key
- **Non prime attributes- {B, C, D, E}**

Since, there is single attribute in candidate key i.e. A, so all the dependencies will be fully functional dependent. No need to check.

So, given relation R is in 2 NF already

3rd Normal Form Example 1

Given: { A->BCDE, C->D, B->E }

Check for 3NF:

- **A->B**

  B is directly dependent on A, thus there is no transitivity. // ***So it is in 3NF***

- **A->C**

  C is directly dependent on A, thus there is no transitivity. // ***So it is in 3NF***

- **A->D**

  D is Transitive dependent on C { A->C & C->D} ***//So not in 3NF***

- **A->E**

  {Transitive dependent: A->B & B->E} ***//So not in 3NF***

Since given relation is not in 3NF. So decompose it-

R1 (A, B, C)	Attributes that follow 3NF with candidate key
R2 (C, D)	Attribute that is transitive dependent with attribute upon which it is transitive dependent
R3 (B, E)	

3rd Normal Form Example 2

Example-2:

Normalize a relation R (A, B, C, D, E, F) up to 3NF, when following functional dependencies are given { AB->C, B->E, D->F, AB->D }

Solution:

Check for 1NF:

- Relation is already in 1 Normal Form, since no records are given in tabular format so assume that relation R (A,B,C,D,E) is having all atomic values.

Check for 2NF:

- Find the candidate keys of R

  (AB)+= {A, B, C, D, E, F}

- **Prime attributes- {A, B }** //Part of candidate key
- **Non prime attributes- { C, D, E, F}**

- **AB->C** **Fully Functional Dependent // *So it is in 2NF***
- **AB->D** **Fully Functional Dependent // *So it is in 2NF***
- **AB->E** **Partial Functional Dependent {B->E} // *So it is not in 2NF***
- **AB->F** **Fully Functional Dependent // *So it is in 2NF***

3rd Normal Form Example 2

Given { AB->C, B->E, D->F, AB->D }

So given relation is not in 2 NF. To covert into 2NF, decompose it-

R1 (A, B, C, D, F) **R2(B, E)**

Now check for dependency preservation-

R1 (A, B, C, D, F)

F1= { AB->C, D->F, AB->D }

R2(B, E)

F2={B->E}

R2 is already in 3NF, B is candidate key & E is directly depend on B

Check 3NF for R1

Candidate key= {AB}

- **AB->C** No Transitive Dependent *//So it is in 3NF*
- **AB->D** No Transitive Dependent *//So it is in 3NF*
- **AB->F** Transitive Dependent {AB->D & D->F *//So it is not in 3NF*

Since R1 is not in 3NF, so decompose it-

R11 (A, B, C, D) & R12 (D, F)

So final decompositions of R-

R11 (A, B, C, D)
R12 (D, F)
R2(B, E)

3rd Normal Form Example 3

Example: Find the highest normal form of a relation R(A, B, C, D, E) with FD set as: { BC->D, AC->BE, B->E }

Explanation:

Step-1: Candidate Key: (AC)+ ={A, C, B, E, D}

Step-2: Prime attributes= {A, C} and non-prime= {B, D, E}

Step-3:

- The relation R is in 1st normal form as a relational DBMS does not allow multi-valued or composite attribute.
- The relation is in 2nd normal form because BC->D is in 2nd normal form (BC is not a proper subset of candidate key AC) and AC->BE is in 2nd normal form (AC is candidate key) and B->E is in 2nd normal form (B is not a proper subset of candidate key AC).
- The relation is not in 3rd normal form because in BC->D (neither BC is a super key nor D is a prime attribute) and in B->E (neither B is a super key nor E is a prime attribute) but to satisfy 3rd normal for, either LHS of an FD should be super key or RHS should be prime attribute.

So the highest normal form of relation will be 2nd Normal form.

Boyce and Codd Normal Form (BCNF)

- After the application of the 2NF and 3NF, some dependencies can still exist that will cause redundancy to be present in relations.
- Although, 3NF is adequate normal form for relational database, still, this (3NF) normal form may not remove 100% redundancy because of X?Y functional dependency, if X is not a candidate key of given relation.
- This weakness in 3NF, resulted in the presentation of a stronger normal form called **Boyce–Codd Normal Form** (Codd, 1974).
- Boyce and Codd Normal Form is a higher version of the Third Normal form. It is stricter than 3NF. This form deals with certain type of anomaly that is not handled by 3NF.

For a table to be in BCNF, following conditions must be satisfied:

"A relation is in BCNF iff, for every functional dependency (X->Y), X must be a super key of the given relation."

Boyce and Codd Normal Form (BCNF)-Example

Example: Let's assume there is a company where employees work in more than one department.

EMPLOYEE

EMP_ID	EMP_COUNTRY	EMP_DEPT	DEPT_TYPE	EMP_DEPT_NO
264	India	Designing	D394	283
264	India	Testing	D394	300
364	UK	Stores	D283	232
364	UK	Developing	D283	549

In the above table Functional dependencies are as follows:

EMP_ID → EMP_COUNTRY

EMP_DEPT → {DEPT_TYPE, EMP_DEPT_NO}

Candidate key: {EMP-ID, EMP-DEPT}

The table is not in BCNF because neither EMP_DEPT nor EMP_ID alone are keys.

To convert the given table into BCNF, we decompose it.

Boyce and Codd Normal Form (BCNF)-Example Cont.....

EMPLOYEE

EMP_ID	EMP_COUNTRY	EMP_DEPT	DEPT_TYPE	EMP_DEPT_NO
264	India	Designing	D394	283
264	India	Testing	D394	300
364	UK	Stores	D283	232
364	UK	Developing	D283	549

EMP_COUNTRY

EMP_ID	EMP_COUNTRY
264	India
264	India
364	UK
364	UK

EMP_DEPT

EMP_DEPT	DEPT_TYPE	EMP_DEPT_NO
Designing	D394	283
Testing	D394	300
Stores	D283	232
Developing	D283	549

EMP_DEPT_MAPPING

EMP_ID	EMP_DEPT
264	Designing
264	Testing
364	Stores
364	Developing

Candidate keys:

For the first table: EMP_ID

For the second table: EMP_DEPT

For the third table: {EMP_ID, EMP_DEPT}

Now, this is in BCNF because left side part of both the functional dependencies is a key.

Boyce and Codd Normal Form (BCNF)-Example 1

Example 1: consider a relation R(A, B, C) and functional dependencies- {A-> BC, B -> A}. Check whether R is in BCNF or not?

Solution:

Given, **FDs= {A->BC, B->A}**

Candidate Keys of R- **{A & B}**

Relation is already in 3 NF, since there is no transitive dependency.

Now, Check for BCNF-

- For dependency A->BC, A is super key/candidate key so it is in BCNF.
- For dependency B->A, B is super key/candidate key so it is also in BCNF.

Thus, A and B both are super keys so above relation is in BCNF.

Boyce and Codd Normal Form (BCNF)-Example 2

Example 2: consider a relation R(A, B, C, D) and functional dependencies- {A-> BCD, BC -> AD, D->B}. Check whether R is in BCNF or not?

Solution:

Given, **FDs= {A-> BCD, BC -> AD, D->B}**

Candidate Keys of R- **{A & BC}**

Relation is already in 3 NF, since there is no transitive dependency.

Now, Check for BCNF-

- A->BCD //*A is super key/candidate key so it is in BCNF.*
- BC->AD //*BC is super key/candidate key so it is in BCNF.*
- D->B //*D is not super key/candidate key so it is not in BCNF.*

Decompositions are- **R1 (A, C, D) & R2 (B, D)**

Boyce and Codd Normal Form (BCNF)-Example 3

Example 3: consider a relation R(A, B, C, D) and functional dependencies- {AB-> CD, D->B}. Check whether R is in BCNF or not?

Solution:

Given, **FDs= {AB-> CD, D->B}**

Candidate Keys of R- **{AB}**

Relation is already in 3 NF, since there is no transitive dependency.

Now, Check for BCNF-

- AB->CD //*AB is super key/candidate key so it is in BCNF.*
- D->B //*D is not super key/candidate key so it is not in BCNF.*

Decompositions are- **R1 (A, C, D) & R2 (B, D)**

Boyce and Codd Normal Form (BCNF)-Example 4

Example 4: consider a relation R(A, B, C) and functional dependencies- {AB-> C, C->B}. Check whether R is in BCNF or not?

Solution:

Given, **FDs= {AB-> C, C->B }**

Candidate Keys of R- **{AB}**

Relation is already in 3 NF, since there is no transitive dependency.

Now, Check for BCNF-

- AB->C //*AB is super key/candidate key so it is in BCNF.*
- C->B //*C is not super key/candidate key so it is not in BCNF.*

Decompositions are- **R1 (A, B) & R2 (B, C)**

4th Normal Form

A table is said to be in the Fourth Normal Form when,

- It is in the Boyce-Codd Normal Form.
- And, it doesn't have Multi-Valued Dependency.

OR

A relation is in 4NF, if it is in BCNF and for every non trivial multivalued functional dependency {X -> -> Y}, X should be the super key of R.

What is multivalued dependency ?

Three conditions for multivalued dependency (A->->B)-

1. For single value of A, more than one value of B exist
2. Table should have at least 3 columns i.e. R(A,B,C)
3. In R(A,B,C), B and C should be independent

If all above three are true then a table is having multivalued dependency.

ENROLMENT TABLE

s_id	course	hobby
1	Science	Cricket
1	Maths	Hockey
2	C#	Cricket
2	Php	Hockey

What problem may arise due to multivalued dependency ?

What is multivalued dependency ?

since there is no relation between course and hobby, so it may create two more additional rows which may be a bad design-

ENROLMENT TABLE

s_id	course	hobby
1	Science	Cricket
1	Maths	Hockey
1	Science	Hockey
1	Maths	Cricket

Solution for 4th normal form is-

CourseOpted TABLE

s_id	course
1	Science
1	Maths
2	C#
2	Php

Hobbies TABLE

s_id	hobby
1	Cricket
1	Hockey
2	Cricket
2	Hockey

4th Normal Form Example 1-

Lets consider a relation Employee (Ename, Pname , Dname) which has following non trivial MVD's:

Ename -> -> Pname

Ename -> -> Dname

Ename	Pname	Dname
Jay	P1	D1
Jay	P1	D2
Jay	P2	D1
Jay	P2	D2

Solution:

Ename is not the super key of the relation. Hence the relation is not in 4NF.

If the relation is decomposed into two relations-

R1 (Ename Pname) , Ename -> -> Pname

Ename	Pname
Jay	P1
Jay	P2

R2 (Ename Dname) , Ename -> -> Dname

Ename	Dname
Jay	D1
Jay	D2

Now in R1 & R2, Ename is again not the super key, but as Ename and Pname together make the relation, and similarly Ename and Dname together make the relation, so these are the trivial Multivalued Dependencies and hence these two relations are in 4NF.

4th Normal Form Example 2-

Convert the following relation into 4NF-

Emp_Name	Skills	Languages
Smith	Cook	German
Smith	Cook	French
Smith	Cook	Greek
Smith	Typist	German
Smith	Typist	French
Smith	Typist	Greek

Solution:

R1 (Emp_Name ->-> Skills)

Emp_Name	Skills
Smith	Cook
Smith	Typist

R2 (Emp_Name ->-> Languages)

Emp_Name	Languages
Smith	German
Smith	French
Smith	Greek

4th Normal Form Example 3-

Convert the following relation into 4NF-

Emp_Name	Project	Degree
John	Online Bus Ticket	B.Tech.
John	Online Booking	B.Tech.
John	Online Bus Ticket	MBA
John	Online Booking	MBA
Smith	Cineplex Ticket	B.Tech.
Smith	Cineplex Ticket	M.Tech.

Solution:

R1 (Emp_Name ->-> Project)

Emp_Name	Project
John	Online Bus Ticket
John	Online Booking
Smith	Cineplex Ticket

R2 (Emp_Name ->-> Degree)

Emp_Name	Degree
John	B.Tech.
John	MBA
Smith	B.Tech.
Smith	M.Tech.

5th Normal Form

- A relation is in 5NF if it is in 4NF and not contains any join dependency.

or

- A relation is in 5NF iff it is in 4NF and it can not be further non loss decomposed.

5NF is satisfied when all the tables are broken into as many tables as possible in order to avoid redundancy.

5NF is also known as Project-join normal form (PJ/NF).

Join Dependency- Let R be a relational schema and R1,R2,R3……..Rn be the decompositions of R, R is said to satisfy join dependency if and only if

$\prod_{R1}(R) \bowtie \prod_{R2}(R) \bowtie \prod_{R3}(R) \bowtie \ldots\ldots \prod_{Rn}(R) = R$

5th Normal Form

<Employee>

EmpName	**EmpSkills**	**EmpJob** (Assigned Work)
Tom	Networking	EJ001
Harry	Web Development	EJ002
Katie	Programming	EJ002

The above table can be decomposed into the following three tables; therefore it is not in 5NF:

<EmployeeSkills>

EmpName	**EmpSkills**
Tom	Networking
Harry	Web Development
Katie	Programming

<EmployeeJob>

EmpName	**EmpJob**
Tom	EJ001
Harry	EJ002
Katie	EJ002

<JobSkills>

EmpSkills	**EmpJob**
Networking	EJ001
Web Developmen	EJ002
Programming	EJ002

Our Join Dependency −

{(EmpName, EmpSkills), (EmpName, EmpJob), (EmpSkills, EmpJob)}

The above relations have join dependency, so they are not in 5NF. That would mean that a join relation of the above three relations is equal to our original relation **<Employee>**.

Why we need a KEY in DBMS?

- **KEYS in DBMS** is an attribute or set of attributes which helps you to identify a row(tuple) in a relation(table).
- They allow you to find the relation between two tables.
- Keys help you to identify any row of data in a table. A table may contain thousands of records & those records could be duplicated. Keys ensure that you can uniquely identify a record.
- Help you to enforce identity and integrity in the relationship.

Thank You